

Lecture Note 1

Cyber Differential Games, PMP, RL/DRL, MADRL, Neural ODE, PINN/PIDL,
and Hybrid Control

A self-contained guide from models to implementation

Prepared for advanced research discussion and supervision

Contents

1	How This Lecture Note Is Organized	2
2	Notation and Method Selection	2
2.1	What each method consumes and outputs	2
3	Game Concepts Before Learning: What Exactly Is Being Solved?	3
3.1	A compact taxonomy for cyber differential games	3
3.2	Nash equilibrium in continuous time and in Markov games	4
3.3	How the HODGE propaganda-war paper fits this taxonomy	5
4	Cyber Dynamic Models: What Is Being Controlled?	6
4.1	Controlled malware propagation	6
4.2	APT repair and deception dynamics	6
4.3	Rumor and misinformation defense	7
4.4	How to read these models operationally	7
4.5	What makes a cyber model suitable for learning-based control?	8
5	Continuous, Discrete, Impulsive, and Hybrid Cyber Control	8
5.1	Continuous-time state dynamics and continuous control	8
5.2	Discrete decision with continuous flow: no jump	8
5.3	Impulsive control: discrete decision with state jump	9
5.4	Hybrid control: discrete mode plus continuous intensity	10
5.5	Mixed continuous-discrete differential games	10
5.6	Which method should be used?	11
6	From Continuous-Time Control to MDPs and Markov Games	11
6.1	Original continuous-time optimal control problem	12
6.2	Step-by-step conversion to an MDP	12
6.3	Discounting and continuous-time costs	13
6.4	Action-to-control map	13
6.5	A practical environment interface	13
6.6	Conversion to a Markov game	14
6.7	What is preserved and what is lost?	14
7	PMP and FBSM: Classical Continuous-Time Solution	15

7.1	PMP conditions	15
7.2	Malware example: deriving the control formula	15
7.3	Forward-backward sweep method	15
7.4	FBSM versus RL/DRL: what changes?	16
8	How PMP Should Be Connected to Learning	16
8.1	Five levels of coupling	16
8.2	Why recent papers often use Level 1	17
9	RL and DRL in Cyber Control	17
9.1	Why use RL?	17
9.2	DQN for discrete cyber actions	18
9.3	Actor-critic methods for continuous controls	18
9.4	DQN training loop in detail	18
9.5	When to use DQN, PPO, SAC, or DDPG	19
9.6	Reward shaping without changing the problem too much	19
10	MARL for Attacker-Defender Interaction	19
10.1	Markov game implementation	19
10.2	Concrete attacker-defender malware/deception game	20
10.3	Training order	20
10.4	Independent learning versus CTDE	21
10.5	Self-play curriculum for cyber games	21
10.6	MADRL algorithm families for attacker-defender games	21
10.7	A step-by-step MADRL recipe for a cyber differential game	22
10.8	Equilibrium evaluation in MADRL	22
10.9	How PMP/FBSM baselines strengthen MADRL papers	23
11	Neural ODE, PINN, and PIDL: What They Do Differently	23
11.1	Neural ODE as a dynamics learner	23
11.2	PINN/PIDL as equation-constrained learning	24
11.3	PMP-informed PINN	24
11.4	How Neural ODE is trained in a cyber setting	24
11.5	When PINN/PIDL is preferable to Neural ODE	25
12	Hybrid Control and Mixed Continuous-Discrete Differential Games	25
12.1	Why hybrid actions are natural	25

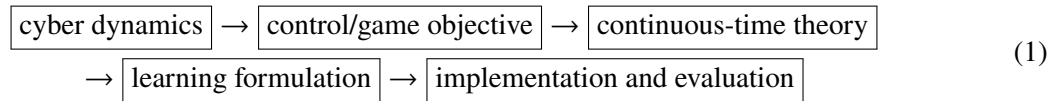
12.2	Solution routes	25
12.3	How to choose among hybrid solution routes	26
12.4	Example hybrid action mapping	26
13	Detailed Case Studies	27
13.1	Case 0: HODGE-style hybrid propaganda war game	27
13.2	Case A: malware defense by DDQN	27
13.3	Case B: APT defense by PMP-informed PINN	28
13.4	Case C: misinformation defense by sequential PINN plus optimal control	28
13.5	Case D: hybrid APT game by PINN/PIDL plus MARL	28
14	How to Report Results	29
15	Implementation Cookbooks for Cyber Control and Games	29
15.1	Cookbook A: FBSM for continuous malware control	29
15.2	Cookbook B: DQN/DDQN for discrete cyber defense	30
15.3	Cookbook C: PPO, DDPG, and SAC for continuous control	31
15.4	Cookbook D: MARL and CTDE for attacker-defender games	32
15.5	Cookbook E: Hybrid continuous-discrete action games	33
15.6	Reward shaping without changing the research objective	33
15.7	Testing the environment before training	33
16	Extra Implementation Guidance: From Equations to Experiments	34
16.1	A minimal ODE-RL environment: objects and data flow	34
16.2	What the RL state should include	34
16.3	Reward scale and sign checks	34
16.4	ODE solver choice in RL experiments	35
16.5	How to document an RL experiment	35
16.6	Specific implementation choices for Section 10 cookbooks	36
16.7	Connecting PMP more tightly to the RL implementation	36
17	Practical Python Implementation Roadmap	37
18	Main Takeaways	37
19	Python Implementation Map and Code Organization	38
A	Deep Learning Background: Networks, Activations, and Optimization	38

A.1	Sigmoid, tanh, and softmax	39
A.2	Why scaling matters	39
B	ODE Solver Background for Cyber Control	39
B.1	Euler and RK4	39
B.2	Adaptive solvers	40
B.3	Conservation and positivity	40
C	MDP, Bellman Equation, and Q-Learning Background	40
C.1	DQN and DDQN details	41
C.2	Practical DQN hyperparameters	41
D	PPO and Continuous-Action Actor-Critic Background	41
D.1	DDPG and SAC	41
E	Markov Game and MARL Background	42
E.1	Independent learning and nonstationarity	42
E.2	Centralized training, decentralized execution	42
F	Hybrid Actions, Softmax Relaxation, and Action Masks	42
F.1	Action masks	42
F.2	Differentiable relaxation	43
G	Appendix: Game-Theoretic Background for MARL and MADRL	43
G.1	Normal-form game, dynamic game, differential game, and Markov game	43
G.2	Zero-sum and general-sum learning objectives	43
G.3	Open-loop, feedback, and sampled-data interpretations	44
G.4	MADDPG-style centralized critic	44
G.5	MAPPO-style centralized value function	44
G.6	Common MADRL failure modes in cyber games	44
H	Appendix: Detailed Conversion of Continuous, Impulsive, and Hybrid Models to RL	45
H.1	Case 1: continuous flow only	45
H.2	Case 2: impulse followed by continuous flow	45
H.3	Case 3: hybrid discrete mode plus continuous intensity	46
H.4	Case 4: attacker-defender Markov game	46
H.5	Common mistakes	46
H.6	Python skeleton: impulse and hybrid cyber environment	47

I	Python Skeletons for ODE-RL and DDQN	48
I.1	Minimal RK4-based cyber defense environment	48
I.2	Environment sanity checks before training	49
I.3	DQN network and DDQN target update	50
I.4	Minimal training loop structure	50
I.5	Extension to attacker-defender Markov games	51

1. How This Lecture Note Is Organized

The note is organized around a clean workflow:



The most important improvement is that RL, DRL, and MARL are not presented as unrelated tools after PMP. Instead, the note explains exactly how a continuous-time optimal control or differential game is converted into an MDP or Markov game, and what information from PMP is preserved or lost in that conversion.

Key methodological principle. PMP and differential games define the continuous-time mathematical problem. RL/DRL/MARL usually solve a discretized feedback-control problem derived from that problem. PINN/PIDL can either learn unknown dynamics, solve the PMP residual system, or approximate HJB/HJI equations. These are different levels of coupling, and they should not be confused.

2. Notation and Method Selection

This note uses a small number of repeated symbols. The same notation is used across optimal control, differential games, RL, and PINN/PIDL so that the link among methods is visible.

Symbol	Meaning
$x(t)$	continuous-time cyber state, for example malware compartments, APT asset states, or misinformation populations
$u_D(t), u_A(t)$	continuous-time defender and attacker controls
a_D^k, a_A^k	discrete-time actions selected at decision epoch t_k
θ	physical or propagation parameters, such as infection, repair, forgetting, deception-decay, or correction rates
J_D, J_A	cumulative defender and attacker objectives in continuous time
H	Hamiltonian used by PMP or differential-game optimality conditions
$\lambda(t)$	costate or adjoint variable; it measures marginal value of changing the state
s_k	RL/DRL/MARL state at decision epoch t_k ; usually derived from $x(t_k)$ and observations
r_k	reward over the interval $[t_k, t_{k+1}]$, usually negative running cost
π	policy mapping states to actions, e.g., $\pi_D(a_D s)$

2.1 What each method consumes and outputs

A useful way to avoid confusion is to ask what each method consumes as input and what it returns as output.

Method	Main input	Main output
PMP	ODE model, cost functional, constraints	optimality system: state, costate, stationarity, boundary conditions
FBSM	PMP optimality system and initial guess for controls	open-loop trajectory $u^*(t)$ and state $x^*(t)$
RL/DQN	MDP simulator (s, a, r, s') with discrete actions	feedback policy or Q-function $Q(s, a)$
PPO/SAC/DDPG	simulator with continuous or stochastic actions	feedback actor $\pi(a s)$ and critic
MARL	Markov game simulator with multiple agents	attacker and defender policies, often with centralized critics
Neural ODE	observed trajectories and differentiable ODE solver	learned dynamics $\dot{x} = f_\theta(x, u, t)$ or residual term g_θ
PINN/PIDL	data, equations, constraints, priors, possibly PMP/HJI residuals	states, parameters, controls, costates, or value functions satisfying residual constraints

Practical selection rule. Use PMP/FBSM when the model is known, continuous, and low- to moderate-dimensional. Use RL/DRL when actions are operational decisions and feedback policies are needed. Use MARL when the attacker adapts. Use Neural ODE when the dynamics are partly unknown. Use PINN/PIDL when equations and constraints are important and data are sparse, or when one wants a neural solver for PMP/HJB/HJI residual systems.

3. Game Concepts Before Learning: What Exactly Is Being Solved?

A source of confusion in recent learning-for-differential-games papers is that the word “game” may refer to several different mathematical objects. Before choosing PMP, FBSM, RL, DRL, MARL, MADRL, PINN, or PIDL, it is useful to identify the game type and the solution concept. Differential games extend optimal control from one decision maker to multiple strategic decision makers: each player controls the same evolving state, but each player evaluates the outcome through its own objective. This point is emphasized in classical dynamic-game theory [31, 32] and in cyber differential-game tutorials.

3.1 A compact taxonomy for cyber differential games

The following table is intended as a practical checklist for malware, APT, deception, and propaganda-war models.

Dimension	Meaning	Typical cyber interpretation
-----------	---------	------------------------------

Zero-sum vs general-sum	In a zero-sum game, $J_A = -J_D$. In a general-sum game, each player has a different payoff.	Malware attacker-defender games may be approximated as zero-sum. Propaganda, compliance, and insider-assisted APT games are often general-sum because costs, reputational effects, and operational benefits are not exact negatives.
Deterministic vs stochastic	The state equation is an ODE or a stochastic process.	Most propagation models use deterministic ODEs for tractability. RL/MADRL can add stochasticity through random initial states, random attacks, noisy observations, or network sampling.
Nash vs Stackelberg	In Nash games, players decide simultaneously. In Stackelberg games, a leader commits first and a follower responds.	Attacker-defender malware control is often modeled as Nash. Security investment, auditing, and compliance guidance can be Stackelberg if the organization commits first and employees or attackers respond.
Open-loop vs feedback	Open-loop controls are functions of time. Feedback controls are functions of the observed state.	PMP/FBSM usually returns $u_i(t)$ for a fixed initial state. RL/MADRL usually returns $\pi_i(a_i s)$ that adapts to the current state.
Continuous vs impulsive vs hybrid	Continuous controls act through $\dot{x} = f(\cdot)$. Impulses create jumps $x(\tau^+) = G(x(\tau^-), v)$. Hybrid models combine continuous flows, discrete modes, and impulses.	Continuous propaganda spending, continuous patching, or monitoring affect transition rates. Human incident response, media-literacy campaigns, isolation, and takedown actions may be better modeled as impulses.

Why this taxonomy matters for learning. DQN, PPO, SAC, MADDPG, MAPPO, and other MADRL methods do not directly require the original PMP equations. They require an environment. The game taxonomy determines what the environment should contain: one agent or multiple agents; zero-sum or general-sum rewards; simultaneous or sequential actions; discrete, continuous, or hybrid actions; continuous flow only or impulse jumps.

3.2 Nash equilibrium in continuous time and in Markov games

For a two-player continuous-time differential game,

$$\dot{x}(t) = f(x(t), u_1(t), u_2(t), t), \quad x(0) = x_0, \quad (2)$$

with objectives $J_1(u_1, u_2)$ and $J_2(u_1, u_2)$, an open-loop Nash equilibrium (u_1^*, u_2^*) satisfies

$$J_1(u_1^*, u_2^*) \geq J_1(u_1, u_2^*) \quad \text{for all } u_1, \quad (3)$$

$$J_2(u_1^*, u_2^*) \geq J_2(u_1^*, u_2) \quad \text{for all } u_2, \quad (4)$$

when the convention is payoff maximization. If the problem is written as cost minimization, the inequality directions are reversed. PMP gives necessary conditions for such equilibria by introducing one Hamiltonian and one costate system for each player. In nonzero-sum games these conditions are normally necessary rather than sufficient, so numerical comparison against alternative strategies remains important.

After time discretization, state observation, action selection, and ODE/impulse simulation, the same strategic idea becomes a Markov game:

$$\mathcal{G} = (\mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, P, \{r_i\}_{i \in \mathcal{N}}, \gamma). \quad (5)$$

Here $\mathcal{N}$ is the player set, $\mathcal{S}$ is the state space, $\mathcal{A}_i$ is player i 's action space, $P(s'|s, a_1, \dots, a_N)$ is the transition kernel, r_i is the reward of player i , and γ is a discount factor. A Markov-perfect Nash equilibrium is a profile of feedback policies $\pi_i^*(a_i|s)$ such that no player can improve its value by unilateral deviation:

$$V_i^{\pi_i^*, \pi_{-i}^*}(s) \geq V_i^{\pi_i, \pi_{-i}^*}(s), \quad \forall \pi_i, \forall s. \quad (6)$$

This is the precise object that MARL and MADRL try to approximate.

3.3 How the HODGE propaganda-war paper fits this taxonomy

The HODGE/ORIENTATION framework is a useful example because it is not a simple continuous-control game. It combines a degree-based network model for dual competitive information spreading with a dual hybrid control mechanism: continuous-time propaganda investment and discrete-time impulsive counter-propaganda investment [18]. In words, propaganda is modeled as persistent influence over an interval, while counter-propaganda is modeled as targeted interventions at selected moments. This distinction is exactly the distinction between continuous flow control and impulse/jump control.

A simplified abstraction is

$$\dot{x}(t) = f(x(t), u_R(t), u_B(t)), \quad t \notin \tau_R \cup \tau_B, \quad (7)$$

$$x(\tau_i^+) = G_R(x(\tau_i^-), v_R(\tau_i)) \quad \text{or} \quad x(\tau_i^+) = G_B(x(\tau_i^-), v_B(\tau_i)), \quad (8)$$

where u_R, u_B are continuous propaganda investment rates and v_R, v_B are impulsive counter-propaganda investment levels. The original HODGE algorithm approximates a Nash strategy profile by iteratively updating continuous and impulsive decisions. A learning-based extension would not simply “replace PMP by MADRL.” Instead, it would define a Markov game in which each party observes a state summary, selects continuous and/or impulsive investment actions at decision epochs, applies jump maps if an impulse is selected, integrates the degree-based ODE until the next decision epoch, and receives a reward based on narrative benefit minus investment cost.

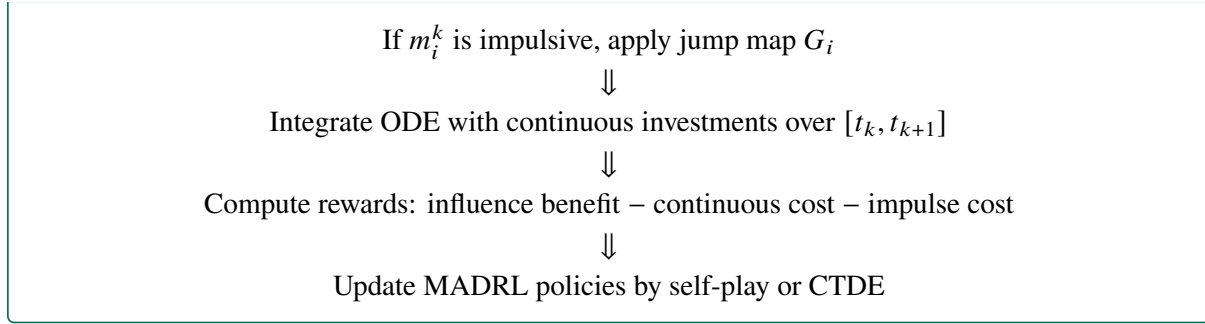
Hybrid propaganda game as a Markov game

Observe degree-based state summary s_k

⇓

Red chooses (m_R^k, v_R^k, u_R^k) , Blue chooses (m_B^k, v_B^k, u_B^k)

⇓



4. Cyber Dynamic Models: What Is Being Controlled?

4.1 Controlled malware propagation

A basic malware or virus propagation model is

$$\dot{S}(t) = -\beta S(t)I(t) - u_p(t)S(t) + \omega R(t), \quad (9)$$

$$\dot{I}(t) = \beta S(t)I(t) - \gamma I(t) - u_c(t)I(t), \quad (10)$$

$$\dot{R}(t) = \gamma I(t) + u_p(t)S(t) + u_c(t)I(t) - \omega R(t). \quad (11)$$

Here $S(t)$ denotes vulnerable devices, $I(t)$ compromised devices, and $R(t)$ recovered or protected devices. The infection term βSI expresses the idea that vulnerable devices are compromised through interaction with infected devices. The control $u_p(t)$ represents patching or hardening, and $u_c(t)$ represents cleaning, monitoring, or incident response.

This model is suitable for introducing optimal control because the defender must balance two competing goals: reduce $I(t)$ and avoid excessive control cost. A typical objective is

$$J_D(u_p, u_c) = \int_0^T \left[A_I I(t) + \frac{B_p}{2} u_p^2(t) + \frac{B_c}{2} u_c^2(t) \right] dt + A_T I(T). \quad (12)$$

The term $A_I I(t)$ is security loss, while $B_p u_p^2/2$ and $B_c u_c^2/2$ penalize aggressive intervention. The terminal term $A_T I(T)$ penalizes residual compromise.

4.2 APT repair and deception dynamics

APT attacks are persistent and stealthy, so it is useful to include repair, isolation, and deception. A compact APT-oriented model is

$$\dot{V} = -\beta(1 - \chi z)VC - u_h V + \omega P, \quad (13)$$

$$\dot{C} = \beta(1 - \chi z)VC - (\gamma + u_r + u_i)C, \quad (14)$$

$$\dot{P} = u_h V + (\gamma + u_r + u_i)C - \omega P, \quad (15)$$

$$\dot{z} = u_d(1 - z) - \xi z. \quad (16)$$

Here V is vulnerable but uncompromised assets, C is compromised assets, P is protected or repaired assets, and $z \in [0, 1]$ is deception effectiveness. The factor $(1 - \chi z)$ reduces the effective compromise rate. When z is large, honeypots, decoys, moving-target defense, or false information confuse the attacker.

This extends differential-game APT repair ideas such as Yang et al. [17] and connects naturally with impulsive and hybrid APT defense models [19, 20].

4.3 Rumor and misinformation defense

For rumor, misinformation, or cyber propaganda, a simple correction model is

$$\dot{U} = -\beta_M UM - \beta_C UC, \quad (17)$$

$$\dot{M} = \beta_M UM - \alpha MC - \delta M - u_f M, \quad (18)$$

$$\dot{C} = \beta_C UC + \alpha MC + u_f M - \delta_C C, \quad (19)$$

$$\dot{R} = \delta M + \delta_C C. \quad (20)$$

U is unaware users, M misinformation believers or spreaders, C correction believers or counter-message spreaders, and R inactive or removed users. The control $u_f(t)$ represents fact-checking, trusted correction, platform intervention, or public communication. This model makes sense for cyber propaganda games, where opposing sides can strategically influence narrative populations [18].

Common structure. Malware, APT, and misinformation models all have the form

$$\dot{x} = f(x, u_D, u_A, \theta, t),$$

where x is a cyber/social state, u_D is defense intervention, u_A is attack effort, and θ contains propagation parameters. This common structure is why PMP, FBSM, HJB/HJI, RL/DRL/MARL, Neural ODE, and PINN/PIDL can be studied in one framework.

4.4 How to read these models operationally

The compartment variables should not be interpreted as only biological analogies. In cybersecurity, a compartment can represent a proportion of devices, accounts, hosts, services, narratives, or users. For example, $I(t)$ in a malware model can be the fraction of compromised IoT devices, while $M(t)$ in a misinformation model can be the fraction of users currently spreading a false narrative. A control is any costly intervention that changes transition rates. Patching changes the transition from vulnerable to protected. Cleaning changes the transition from compromised to recovered. Fact-checking changes the transition from misinformation to correction. Deception changes the effective attack rate by altering the attacker's information.

The main modeling decision is therefore not the name of the compartments but the transition mechanism. A paper should explain why each term exists. For example, βSI means mass-action infection: compromise increases when both vulnerable and infected assets are present. The term $(1 - \chi z)$ means deception reduces useful attacker contact. The term $u_f M$ means correction effort converts misinformation spreaders into corrected or inactive users. Clear interpretation of every term is essential before applying PMP or learning methods.

4.5 What makes a cyber model suitable for learning-based control?

A model is suitable when it can be simulated under candidate actions. For RL/DRL/MARL, the simulator may be an ODE solver, an agent-based simulator, a cyber range, or a hybrid environment. For PINN/PIDL, the model must provide differentiable residuals or differentiable approximations. For Neural ODE, one needs trajectory data or simulated traces to train the dynamics. For PMP/FBSM, the model should be differentiable with respect to states and controls so that H_x , H_u , and costate equations can be derived.

5. Continuous, Discrete, Impulsive, and Hybrid Cyber Control

This section clarifies a point that often causes confusion. In cyber control papers, the words *continuous*, *discrete*, *impulsive*, and *hybrid* refer to different parts of the model. They should not be used interchangeably.

5.1 Continuous-time state dynamics and continuous control

A continuous-time cyber model assumes that the state changes continuously over time:

$$\dot{x}(t) = f(x(t), u(t), t), \quad t \in [0, T]. \quad (21)$$

Here $x(t)$ may be the proportion of vulnerable, infected, compromised, recovered, or misinformed users. A continuous control $u(t)$ is a function defined over time. For example,

$$u(t) = \text{patching intensity at time } t, \quad 0 \leq u(t) \leq 1. \quad (22)$$

This means that the defender can continuously adjust the strength of patching, monitoring, deception, or fact-checking. Mathematically, $u(t)$ may be an open-loop function of time or a feedback law $u(t) = \pi(x(t), t)$.

Important distinction. Continuous control does not necessarily mean the defender manually changes the action at every instant. It means the mathematical decision variable is a function over the whole time interval. In numerical implementation, it is often approximated by a piecewise-constant or piecewise-linear function.

For example, in a malware model,

$$\dot{S} = -\beta SI - u_p(t)S, \quad \dot{I} = \beta SI - (\gamma + u_c(t))I, \quad (23)$$

where $u_p(t)$ is continuous patching intensity and $u_c(t)$ is continuous cleaning intensity. PMP/FBSM is natural for this setting because the unknown is a control trajectory $u(t)$ over $[0, T]$.

5.2 Discrete decision with continuous flow: no jump

In real cyber operations, decisions are usually made at discrete times:

$$t_k = k\Delta t, \quad k = 0, 1, \dots, K. \quad (24)$$

At time t_k , the defender chooses an action a_k , such as **patch**, **monitor**, **isolate**, or **deceive**. After that choice, the system still evolves continuously until the next decision time:

$$\dot{x}(t) = f(x(t), a_k, t), \quad t \in [t_k, t_{k+1}). \quad (25)$$

There is no instantaneous jump in the state:

$$x(t_k^+) = x(t_k^-). \quad (26)$$

Only the vector field changes. For instance, choosing **patch** may increase the patching rate during the next day, but the number of infected machines does not instantly drop at the exact decision instant.

Discrete decision without jump:
observe $x_k \rightarrow$ choose $a_k \rightarrow$ hold a_k over $[t_k, t_{k+1}) \rightarrow$ integrate ODE $\rightarrow$ obtain x_{k+1}

This is the cleanest bridge to RL/DRL. The induced MDP transition is

$$x_{k+1} = \text{ODESolve}(f(\cdot, a_k), x_k, t_k, t_{k+1}). \quad (27)$$

The reward is usually the negative accumulated cost over the same interval:

$$r_k = - \int_{t_k}^{t_{k+1}} L(x(t), a_k) dt. \quad (28)$$

DQN/DDQN can be used if a_k is discrete; PPO/SAC/DDPG can be used if the action includes continuous intensity.

5.3 Impulsive control: discrete decision with state jump

An impulsive model assumes that some interventions cause an instantaneous state change at selected times. The dynamics are then

$$\begin{cases} \dot{x}(t) = f(x(t), u(t), t), & t \neq \tau_j, \\ x(\tau_j^+) = G_j(x(\tau_j^-), a_j), & t = \tau_j. \end{cases} \quad (29)$$

Here τ_j is an impulse time, $x(\tau_j^-)$ is the state just before the impulse, and $x(\tau_j^+)$ is the state just after the impulse. In cybersecurity, impulses are natural when a decision has an immediate effect, for example:

- emergency patching instantly moves a fraction of vulnerable machines to protected machines;
- isolation instantly removes a fraction of compromised hosts from the active network;
- a takedown operation instantly removes malicious accounts or command-and-control nodes;
- a fact-checking campaign instantly shifts a fraction of misinformed users into a corrected class.

A simple malware impulse can be written as

$$\begin{bmatrix} S(\tau_j^+) \\ I(\tau_j^+) \\ R(\tau_j^+) \end{bmatrix} = \begin{bmatrix} (1 - p_j)S(\tau_j^-) \\ (1 - q_j)I(\tau_j^-) \\ R(\tau_j^-) + p_jS(\tau_j^-) + q_jI(\tau_j^-) \end{bmatrix}, \quad (30)$$

where p_j is the emergency patching fraction and q_j is the emergency isolation/cleaning fraction. Impulsive APT defense and defense outsourcing models follow this logic [19, 20].

Impulsive RL transition:

observe pre-jump state $x_k^- \rightarrow$ choose impulse action $a_k \rightarrow$ jump $y_k = G(x_k^-, a_k) \rightarrow$ integrate ODE
 $\rightarrow$ next state x_{k+1}^-

The MDP transition becomes

$$y_k = G(x_k, a_k), \quad x_{k+1} = \text{ODESolve}(f, y_k, t_k, t_{k+1}). \quad (31)$$

The reward should include both the running cost and the impulse cost:

$$r_k = - \int_{t_k}^{t_{k+1}} L(x(t), a_k) dt - C_{\text{imp}}(x_k, a_k). \quad (32)$$

Thus the RL environment must implement both a `jump()` function and an `integrate()` function.

5.4 Hybrid control: discrete mode plus continuous intensity

Hybrid cyber actions combine a discrete mode and a continuous parameter:

$$a_k = (m_k, v_k), \quad m_k \in \{1, \dots, M\}, \quad v_k \in [0, 1]. \quad (33)$$

For example,

$$m_k = \text{patch}, \quad v_k = 0.7 \quad (34)$$

means that the defender chooses the patching mode with intensity 0.7. Hybrid control may affect the system in two ways:

1. Mode-dependent continuous flow:

$$\dot{x} = f_{m_k}(x, v_k, t), \quad t \in [t_k, t_{k+1}). \quad (35)$$

2. Mode-dependent jump plus flow:

$$y_k = G_{m_k}(x_k, v_k), \quad x_{k+1} = \text{ODESolve}(f_{m_k}, y_k, t_k, t_{k+1}). \quad (36)$$

This form is often the most realistic for cyber defense because the defender chooses both *what to do* and *how strongly to do it*.

5.5 Mixed continuous-discrete differential games

In an attacker-defender game, both players may have hybrid actions:

$$a_D^k = (m_D^k, v_D^k), \quad a_A^k = (m_A^k, v_A^k). \quad (37)$$

The continuous flow can be written as

$$\dot{x} = f_{m_D^k, m_A^k}(x, v_D^k, v_A^k, t), \quad t \in [t_k, t_{k+1}). \quad (38)$$

If the actions also cause jumps,

$$y_k = G_{m_D^k, m_A^k}(x_k, v_D^k, v_A^k), \quad x_{k+1} = \text{ODESolve}(f_{m_D^k, m_A^k}, y_k, t_k, t_{k+1}). \quad (39)$$

The induced Markov game has transition

$$P(s_{k+1}|s_k, a_D^k, a_A^k) \quad (40)$$

implemented by jump and ODE integration. Defender and attacker rewards are

$$r_D^k = - \int_{t_k}^{t_{k+1}} L_D(x(t), a_D^k, a_A^k) dt - C_D^{\text{imp}}, \quad (41)$$

$$r_A^k = \int_{t_k}^{t_{k+1}} L_A(x(t), a_D^k, a_A^k) dt - C_A^{\text{imp}}. \quad (42)$$

This is the natural setting for MARL. The continuous-time model still provides the scientific mechanism, but the learning algorithm interacts with a discrete-time Markov game simulator.

5.6 Which method should be used?

Model type	Mathematical form	Suitable learning/control method
Continuous control	$\dot{x} = f(x, u(t), t)$	PMP, FBSM, direct collocation, PMP-informed PINN
Discrete decision without jump	$\dot{x} = f(x, a_k, t)$ over each interval	DQN/DDQN for discrete actions; PPO/SAC/DDPG for continuous actions
Impulsive control	$x^+ = G(x^-, a_k)$ plus continuous flow	impulse control theory; RL with jump+ODE transition; hybrid PMP for baseline
Hybrid action	$a_k = (m_k, v_k)$	parameterized-action RL, hybrid PPO/SAC, relaxed PINN for differentiable approximation
Attacker-defender hybrid game	$\dot{x} = f(x, a_D^k, a_A^k)$ and possibly jumps	Markov game, MARL, CTDE, MAPPO/MADDPG-style training

6. From Continuous-Time Control to MDPs and Markov Games

This section is the bridge that is often missing in papers. It explains how a continuous-time optimal control or differential game becomes a discrete-time learning environment.

6.1 Original continuous-time optimal control problem

A defender-only control problem is

$$\min_{u_D(\cdot)} J_D = \Phi(x(T)) + \int_0^T L_D(x(t), u_D(t), t) dt, \quad (43)$$

$$\text{s.t. } \dot{x}(t) = f(x(t), u_D(t), t), \quad x(0) = x_0. \quad (44)$$

PMP and FBSM solve this problem directly in continuous time. RL and DRL usually solve a derived discrete-time feedback-control problem.

6.2 Step-by-step conversion to an MDP

Choose decision times

$$0 = t_0 < t_1 < \dots < t_K = T, \quad t_{k+1} - t_k = \Delta t. \quad (45)$$

At each t_k , the defender observes a state

$$s_k = \psi(x(t_k), b_k, o_k), \quad (46)$$

where b_k may represent budget and o_k may represent observations such as alerts or detection scores. The defender chooses an action a_k . The action can be discrete, continuous, or hybrid:

$$\text{discrete: } a_k \in \{\text{patch, monitor, isolate, deceive}\}, \quad (47)$$

$$\text{continuous: } a_k = [u_p^k, u_c^k, u_d^k] \in [0, 1]^3, \quad (48)$$

$$\text{hybrid: } a_k = (m_k, v_k), \quad m_k \in \{\text{patch, monitor, deceive}\}, \quad v_k \in [0, 1]. \quad (49)$$

The continuous dynamics then run between decision points:

$$x_{k+1} = \text{ODESolve}(f, x_k, a_k, [t_k, t_{k+1}]). \quad (50)$$

The reward is the negative of the interval cost:

$$r_k = - \int_{t_k}^{t_{k+1}} L_D(x(t), a_k, t) dt. \quad (51)$$

A simple numerical approximation is

$$r_k \approx -\Delta t L_D(x_{k+1}, a_k, t_{k+1}). \quad (52)$$

Now the problem is an MDP

$$(\mathcal{S}, \mathcal{A}, P, r, \gamma_d), \quad (53)$$

where P is implicitly generated by the ODE solver and any stochastic disturbances. This is the correct Level 1 bridge from optimal control to RL [4].

Continuous-time model to MDP

$$\dot{x} = f(x, u, t), J = \int L dt \implies \text{choose } \Delta t \implies \text{define } s_k \text{ and } a_k \implies \text{ODE transition } s_{k+1} \implies \text{reward } r_k \approx - \int L dt \implies \text{train RL/DRL policy } \pi(a|s).$$

6.3 Discounting and continuous-time costs

If the original continuous-time objective uses discounting

$$J = \int_0^T e^{-\rho t} L(x(t), u(t)) dt, \quad (54)$$

then the discrete discount factor can be chosen as

$$\gamma_d = e^{-\rho \Delta t}. \quad (55)$$

If the original problem has a finite horizon and no discount, many implementations simply set γ_d close to one and terminate at $K = T/\Delta t$. The key is to make the discrete reward consistent with the original running cost. Otherwise, the DRL policy may optimize a different problem from the one stated in the differential-game model.

6.4 Action-to-control map

A common source of confusion is the difference between a discrete RL action and a continuous control in the ODE. The bridge is an action-to-control map

$$\Gamma : \mathcal{A} \rightarrow \mathcal{U}, \quad u(t) = \Gamma(a_k), \quad t \in [t_k, t_{k+1}). \quad (56)$$

For example,

$$\Gamma(\text{patch}) = (u_p, u_c, u_d) = (0.8, 0.1, 0), \quad (57)$$

while

$$\Gamma(\text{deceive}) = (0.1, 0.1, 0.9). \quad (58)$$

In hybrid-action RL, $a_k = (m_k, v_k)$ and the map becomes

$$\Gamma(m_k, v_k) = v_k \Gamma_{\max}(m_k), \quad (59)$$

so the agent chooses both intervention type and intensity.

6.5 A practical environment interface

In code, the continuous-time model becomes an environment with two important functions:

ODE-based cyber MDP interface.

1. **reset**: sample or set x_0 , budget b_0 , attack scenario, and horizon K .
2. **observe**: return $s_k = \psi(x_k, o_k, b_k)$, where o_k may be noisy alerts or partial measurements.
3. **step(a_k)**: convert a_k to ODE controls, integrate the dynamics for one interval, compute

reward, update budget, and return $(s_{k+1}, r_k, done)$.

The Markov property requires that s_k contain enough information to predict the next state distribution given the action. If the defender observes only alerts rather than true compromise levels, the problem is closer to a partially observable MDP. A practical workaround is to include a history window, recurrent neural network, or state estimator.

6.6 Conversion to a Markov game

For an attacker-defender differential game,

$$\dot{x} = f(x, u_A, u_D, t), \quad (60)$$

$$J_D = \Phi_D(x(T)) + \int_0^T L_D(x, u_A, u_D, t) dt, \quad (61)$$

$$J_A = \Phi_A(x(T)) + \int_0^T L_A(x, u_A, u_D, t) dt. \quad (62)$$

Discretize time. At step k ,

$$a_A^k \sim \pi_A(\cdot | s_k), \quad a_D^k \sim \pi_D(\cdot | s_k). \quad (63)$$

The transition is

$$s_{k+1} = \text{ODESolve}(f, s_k, a_A^k, a_D^k, [t_k, t_{k+1}]), \quad (64)$$

and rewards are

$$r_D^k \approx -\Delta t L_D(s_{k+1}, a_A^k, a_D^k), \quad (65)$$

$$r_A^k \approx \Delta t L_A(s_{k+1}, a_A^k, a_D^k). \quad (66)$$

This gives a Markov game

$$(\mathcal{S}, \mathcal{A}_A, \mathcal{A}_D, P, r_A, r_D, \gamma_d). \quad (67)$$

MARL learns policies π_A and π_D by repeated interaction.

6.7 What is preserved and what is lost?

This conversion preserves the state dynamics, intervention costs, and strategic interaction. However, it usually does not preserve the costate equations or Hamiltonian stationarity conditions unless these are explicitly added to the learning objective. Therefore, a DRL solver for the induced MDP is not automatically a solver for the PMP boundary-value problem. It is a feedback-policy solver for a discretized control/game problem.

7. PMP and FBSM: Classical Continuous-Time Solution

7.1 PMP conditions

For

$$\min_u \Phi(x(T)) + \int_0^T L(x, u, t) dt, \quad \dot{x} = f(x, u, t), \quad (68)$$

define the Hamiltonian

$$H(x, u, \lambda, t) = L(x, u, t) + \lambda^\top f(x, u, t). \quad (69)$$

PMP gives [1, 2]

$$\dot{x}^* = H_\lambda(x^*, u^*, \lambda, t), \quad (70)$$

$$\dot{\lambda} = -H_x(x^*, u^*, \lambda, t), \quad (71)$$

$$u^*(t) = \arg \min_{u \in \mathcal{U}} H(x^*, u, \lambda, t), \quad (72)$$

$$\lambda(T) = \Phi_x(x^*(T)). \quad (73)$$

For a zero-sum differential game, the saddle-point condition is

$$u_D^* = \arg \min_{u_D} H(x, u_A^*, u_D, \lambda, t), \quad u_A^* = \arg \max_{u_A} H(x, u_A, u_D^*, \lambda, t). \quad (74)$$

7.2 Malware example: deriving the control formula

For the controlled malware model, the Hamiltonian is

$$\begin{aligned} H = & A_I I + \frac{B_p}{2} u_p^2 + \frac{B_c}{2} u_c^2 + \lambda_S (-\beta S I - u_p S + \omega R) \\ & + \lambda_I (\beta S I - \gamma I - u_c I) + \lambda_R (\gamma I + u_p S + u_c I - \omega R). \end{aligned} \quad (75)$$

The stationarity conditions are

$$H_{u_p} = B_p u_p - \lambda_S S + \lambda_R S = 0, \quad (76)$$

$$H_{u_c} = B_c u_c - \lambda_I I + \lambda_R I = 0. \quad (77)$$

With bounds, the controls are

$$u_p^*(t) = \text{clip} \left(\frac{(\lambda_S - \lambda_R) S}{B_p}, 0, u_{p, \max} \right), \quad (78)$$

$$u_c^*(t) = \text{clip} \left(\frac{(\lambda_I - \lambda_R) I}{B_c}, 0, u_{c, \max} \right). \quad (79)$$

This formula is interpretable: if the marginal benefit of moving vulnerable devices to protected devices is high, u_p increases; if the marginal benefit of cleaning compromised devices is high, u_c increases.

7.3 Forward-backward sweep method

The PMP system is a two-point boundary-value problem. FBSM solves it iteratively [3]:

Guess $u^{(0)}(t) \rightarrow$ forward solve $x^{(i)}(t) \rightarrow$ backward solve $\lambda^{(i)}(t) \rightarrow$ update $u^{(i+1)}(t) \rightarrow$ repeat.

A relaxed update is

$$u^{(i+1)}(t) = \alpha \tilde{u}^{(i+1)}(t) + (1 - \alpha)u^{(i)}(t), \quad 0 < \alpha \leq 1. \quad (80)$$

FBSM is an excellent baseline when the dynamics are known and the control is continuous. It becomes less natural when the defender must choose discrete actions such as isolate or deceive.

7.4 FBSM versus RL/DRL: what changes?

FBSM and RL solve different computational versions of the same modeling idea. FBSM normally returns an open-loop control $u^*(t)$ for a fixed initial condition and fixed parameters. If the actual cyber state deviates from the predicted trajectory, the open-loop schedule may be suboptimal unless the problem is solved again. RL/DRL instead learns a feedback policy $\pi(a|s)$ that reacts to the observed state.

Aspect	FBSM/PMP	RL/DRL/MARL
Control type	naturally continuous	discrete, continuous, or hybrid
Output	open-loop trajectory $u(t)$	feedback policy $\pi(a s)$
Theory	strong optimality interpretation under assumptions	approximate, data/simulation-driven
Need derivatives	yes, H_x, H_u	no analytic derivatives needed
Adaptation	requires re-solving or feedback extension	policy directly reacts to state
Discrete modes	awkward unless switching/impulse control is used	natural
Training cost	usually cheap for low-dimensional ODEs	can be expensive due to many episodes
Interpretability	high	medium to low, unless policy is analyzed

A useful experimental design is to treat FBSM as a continuous-control benchmark and DRL as a practical feedback controller. If DRL is claimed to improve FBSM, explain whether the improvement comes from feedback adaptation, discrete action modeling, robustness to stochasticity, or simply from optimizing a different objective.

8. How PMP Should Be Connected to Learning

8.1 Five levels of coupling

Level	Name	Meaning
0	Disconnected learning	PMP is derived, but RL/DRL ignores H , λ , and H_u . It only uses simulator rewards.

1	PMP-formulated DRL	RL/-	PMP/differential-game theory defines the continuous-time model and payoff; RL solves the induced MDP or Markov game. This is common in DDQN malware and PPO deception studies [21, 22, 23].
2	PMP-assisted learning		FBSM/PMP controls are used as expert demonstrations, warm starts, reward shaping, or baselines.
3	PMP-informed N/PIDL	PIN-	The neural loss explicitly contains state residuals, costate residuals, stationarity residuals $\ H_u\ ^2$, and boundary residuals.
4	HJI/Pontryagin operator learning		The learner approximates value gradients or costate operators across states and parameters, as in Pontryagin neural operator work [26].

Writing advice. If a method trains on (s, a, r, s') only, call it a DRL solver for the induced MDP or Markov game. If the loss contains $\dot{\lambda} + H_x, H_u$, or HJI residuals, call it PMP-informed or HJI-informed. This avoids overstating the connection between PMP and learning.

8.2 Why recent papers often use Level 1

PMP/FBSM gives structure but may be too slow, too offline, or too continuous for operational cyber actions. For example, DDQN malware suppression formulates the malware game and then uses DDQN because actions such as patching decisions are discrete [21]. Time-delay deception deployment formulates a differential game but converts it to a Markov game and uses PPO to learn real-time feedback policies [22]. These are valid designs, but the methodological contribution is not “solving PMP by DRL.” It is “using DRL to solve the feedback decision problem induced by a differential game.”

9. RL and DRL in Cyber Control

This section keeps the main ideas concise. Appendix C gives the prerequisite details.

9.1 Why use RL?

RL is useful when the defender can simulate the cyber system but cannot easily solve the analytic optimality system. It is also useful when actions are operational and discrete. In the MDP derived in Section 3, RL learns

$$\pi(a|s) \quad \text{or} \quad Q(s, a), \quad (81)$$

so the defender can react to the current cyber state instead of following a precomputed open-loop control $u(t)$.

9.2 DQN for discrete cyber actions

For discrete actions such as no-op, patch, monitor, isolate, and deceive, DQN approximates $Q(s, a)$ using a neural network [5]. The Bellman target is

$$y = r + \gamma_d \max_{a'} Q_{\theta^-}(s', a'), \quad (82)$$

and the training loss is

$$\mathcal{L}_{DQN}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(Q_{\theta}(s, a) - y)^2]. \quad (83)$$

Double DQN reduces overestimation by using the online network to select the next action and the target network to evaluate it [6]:

$$y^{DDQN} = r + \gamma_d Q_{\theta^-} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right). \quad (84)$$

DQN implementation for malware defense.

1. State: $s_k = [S_k, I_k, R_k, z_k, b_k]$, normalized to $[0, 1]$.
2. Actions: {no-op, patch, monitor, isolate, deceive}.
3. Transition: map the action to ODE parameters, integrate from t_k to t_{k+1} .
4. Reward: $r_k = -\Delta t(A_I I_{k+1} + c_a(a_k) + c_{iso} \mathbf{1}_{a_k=\text{isolate}} I_{k+1})$.
5. Network: MLP $5 \rightarrow 64 \rightarrow 64 \rightarrow 5$.
6. Training: replay buffer, target network, ϵ -greedy exploration, Adam optimizer.

9.3 Actor-critic methods for continuous controls

If actions are continuous intensities such as $u_p, u_c, u_d \in [0, 1]$, DQN is inconvenient. Actor-critic methods learn a policy network and a value or Q network. DDPG learns a deterministic actor for continuous actions [7]; PPO uses a clipped policy-gradient objective for stable updates [8]; SAC adds entropy regularization for exploration [9]. A typical actor for cyber defense is

$$\pi_{\theta}(s) = [u_p(s), u_c(s), u_d(s)], \quad u_i(s) = \text{sigmoid}(h_i(s)). \quad (85)$$

9.4 DQN training loop in detail

DQN learns from transitions rather than complete analytic equations. For an ODE-based malware environment, each transition is generated by solving the ODE for one decision interval. The training loop is:

1. Initialize the online Q-network Q_{θ} and target network Q_{θ^-} .
2. At state s_k , choose a random action with probability ϵ , otherwise choose $a_k = \arg \max_a Q_{\theta}(s_k, a)$.
3. Run the ODE environment to obtain $(s_{k+1}, r_k, done)$.
4. Store $(s_k, a_k, r_k, s_{k+1}, done)$ in replay buffer $\mathcal{D}$.

5. Sample a mini-batch from $\mathcal{D}$ and compute Bellman targets.
6. Update θ by minimizing squared Bellman error.
7. Periodically copy θ to θ^- or use a soft update.

Typical starting hyperparameters are replay buffer size 10^4 – 10^6 , mini-batch size 64–256, learning rate 10^{-4} – 10^{-3} , target update every 100–1000 gradient steps, and an ϵ schedule decreasing from 1 to 0.05. These values should be reported because they affect reproducibility.

9.5 When to use DQN, PPO, SAC, or DDPG

- Use **DQN/DDQN** when actions are a small set of discrete interventions such as patch, isolate, monitor, and deceive.
- Use **PPO** when actions are discrete, continuous, or hybrid and stable policy updates are more important than sample efficiency.
- Use **DDPG/TD3/SAC** when controls are continuous resource intensities. SAC is often more robust because entropy encourages exploration.
- Use **parameterized-action RL** when a decision has both a mode and a continuous intensity, for example patch at intensity 0.7.

A good paper should justify the algorithm from the action space and cyber decision structure, not just use a popular DRL method.

9.6 Reward shaping without changing the problem too much

Reward shaping is useful but dangerous. The base reward should be the negative running cost from the original optimal-control problem:

$$r_k^{base} \approx - \int_{t_k}^{t_{k+1}} L(x(t), a_k) dt. \quad (86)$$

Additional terms can encourage containment or safe exploration, for example

$$r_k = r_k^{base} + \eta_1 (C_k - C_{k+1}) - \eta_2 \mathbf{1}\{C_{k+1} > C_{max}\}. \quad (87)$$

However, if the shaping terms dominate, the DRL agent may solve a different problem. In research writing, list the base objective first and state clearly which shaping terms are added for training stability.

10. MARL for Attacker-Defender Interaction

10.1 Markov game implementation

A Markov game is used when the attacker adapts. At each step,

$$a_A^k \sim \pi_A(\cdot | s_k), \quad a_D^k \sim \pi_D(\cdot | s_k), \quad (88)$$

and the ODE environment returns s_{k+1}, r_A^k, r_D^k . The defender and attacker update their policies using sampled trajectories.

MARL interaction loop

state $s_k \rightarrow$ attacker action a_A^k and defender action $a_D^k \rightarrow$ ODE cyber transition $s_{k+1} \rightarrow$ rewards
 $r_A^k, r_D^k \rightarrow$ update π_A, π_D .

10.2 Concrete attacker-defender malware/deception game

Use $s_k = [V_k, C_k, P_k, z_k]$. Defender actions are patch, monitor, isolate, deceive. Attacker actions are scan, exploit, lateral movement, stealth. The ODE between decision points is

$$\dot{V} = -\beta_k(1 - \chi z)VC - \rho_k V + \omega P, \quad (89)$$

$$\dot{C} = \beta_k(1 - \chi z)VC - (\gamma + \mu_k + \iota_k)C, \quad (90)$$

$$\dot{P} = \rho_k V + (\gamma + \mu_k + \iota_k)C - \omega P, \quad (91)$$

$$\dot{z} = \eta_k(1 - z) - \xi z - \zeta \mathbf{1}_{a_A \in \{\text{scan}, \text{stealth}\}} z. \quad (92)$$

The defender reward is

$$r_D^k = -\Delta t \left(w_C C_{k+1} + c_\rho \rho_k^2 + c_\mu \mu_k^2 + c_\iota \iota_k^2 + c_\eta \eta_k^2 + c_U \iota_k C_{k+1} \right), \quad (93)$$

and the attacker reward is

$$r_A^k = \Delta t \left(q_C C_{k+1} - d_A(a_A^k) - q_z z_{k+1} - q_P P_{k+1} \right). \quad (94)$$

10.3 Training order

A stable training order is:

1. train defender against scripted attacker;
2. train attacker against fixed or pretrained defender;
3. start self-play from pretrained policies;
4. evaluate against unseen attackers and unseen defense costs.

Independent learning is simple but unstable. Centralized training and decentralized execution (CTDE) improves stability by using critics such as

$$Q_D(s, a_A, a_D), \quad Q_A(s, a_A, a_D), \quad (95)$$

while actors execute using local observations. MADDPG and MAPPO are representative CTDE-style methods [11, 12].

10.4 Independent learning versus CTDE

In independent learning, the attacker treats the defender as part of the environment and the defender treats the attacker as part of the environment. This is simple but can be unstable because each agent's learning changes the transition distribution for the other agent. CTDE addresses this by allowing the critic to see joint information during training:

$$Q_D(s, a_A, a_D), \quad Q_A(s, a_A, a_D), \quad (96)$$

while each actor still executes using its own observation. This is appropriate for cyber research because offline training may use complete simulator information, while deployment only uses observable alerts and measurements.

10.5 Self-play curriculum for cyber games

A practical MARL curriculum is:

1. train the defender against a weak scripted attacker;
2. train the defender against stronger scripted attackers;
3. freeze defender and train attacker to exploit it;
4. alternate attacker and defender updates;
5. evaluate against held-out attackers and randomized parameters.

This curriculum is often more stable than starting from fully random self-play. It also produces interpretable experimental stages: basic containment, adaptive attack, and robust defense.

10.6 MADRL algorithm families for attacker-defender games

MARL is the broad field of reinforcement learning with multiple agents. MADRL is the deep-learning version, where policies, critics, or value functions are neural networks. For cyber differential games, four families are especially useful.

Family	How it works	When it is suitable
Independent DQN/PPO	Each agent treats the other agents as part of the environment and learns its own value or policy.	Simple baseline. Useful for small cyber games, but training can be unstable because the environment is nonstationary from each agent's view.
Self-play	Agents repeatedly train against recent or historical versions of opponents.	Good for attacker-defender cyber games where we want adaptive behavior rather than a policy overfitted to one fixed attacker.
Centralized critic, decentralized actors	During training, the critic sees joint actions and possibly global state; during execution, each actor uses local observation.	Useful for simultaneous attacker-defender decisions. MADDPG and MAPPO follow this spirit [11, 12].

Opponent modeling / population training	Agents learn or sample from a population of opponent policies.	Useful when the defender must be ro- bust against many attacker types, not just the latest self-play opponent.
--	---	--

A common CTDE critic for player i is

$$Q_i(s, a_1, \dots, a_N; \phi_i), \quad (97)$$

while the decentralized actor is

$$\pi_i(a_i | o_i; \theta_i), \quad (98)$$

where o_i may be a local observation rather than the full state. In an APT game, the defender may observe alerts and estimated compromise ratios, while the attacker may observe discovered vulnerable assets. The centralized critic is allowed to use the full simulator state during training because it is a training device, not an operational assumption.

10.7 A step-by-step MADRL recipe for a cyber differential game

For a two-player attacker-defender game, use the following sequence.

1. **Start from the continuous-time game.** Write $\dot{x} = f(x, u_A, u_D)$ and objectives J_A, J_D .
2. **Choose the decision interval.** Set $t_k = k\Delta t$. The interval should match operational decision frequency: minutes for automated patching, hours or days for propaganda response.
3. **Define observations.** Use $o_D^k = h_D(x(t_k))$ and $o_A^k = h_A(x(t_k))$. If both agents see the full state, the game is fully observed. If not, the practical model is a partially observed Markov game.
4. **Define actions.** Choose discrete actions, continuous intensities, or hybrid parameterized actions. Example: defender action $a_D = (\text{deceive}, 0.7)$; attacker action $a_A = (\text{lateral movement}, 0.5)$.
5. **Apply impulse first if needed.** If the action is an incident-response impulse, compute $x(t_k^+) = G(x(t_k^-), a_A^k, a_D^k)$.
6. **Integrate the ODE.** Compute $x(t_{k+1}) = \text{ODESolve}(f, x(t_k^+), a_A^k, a_D^k, \Delta t)$.
7. **Compute rewards by interval integration.** For player i , set $r_i^k \approx \int_{t_k}^{t_{k+1}} L_i(x, u_A, u_D) dt$ plus any impulse reward/cost.
8. **Train policies.** Use independent PPO for a simple start, then MAPPO or MADDPG for CTDE.
9. **Evaluate game quality.** Report not only average reward but also best-response tests, exploitability proxies, robustness to unseen opponents, and comparison with PMP/FBSM or HODGE-style baselines.

10.8 Equilibrium evaluation in MADRL

A trained pair of policies is not automatically a Nash equilibrium. In a paper, avoid saying that self-play “finds the Nash equilibrium” unless a meaningful equilibrium diagnostic is provided. Practical diagnostics include:

1. **Unilateral deviation test:** freeze the defender and retrain a new attacker best response. Then freeze the attacker and retrain a new defender best response.

2. **Exploitability proxy:** measure how much reward a best-response agent gains over the trained agent.
3. **Opponent pool robustness:** evaluate the defender against random, static, PMP/FBSM, HODGE-style, and learned attackers.
4. **Cost-effectiveness comparison:** compare cumulative security benefit minus control cost, similar in spirit to hybrid-control propaganda-war experiments [18].
5. **Structural sanity check:** verify that actions make cyber sense, e.g., early containment or deception when compromise grows, reduced spending when the system is already safe.

Important wording. For MADRL results, it is safer to write “the learned policy profile is an approximate equilibrium candidate” or “a robust adaptive strategy profile” unless best-response or exploitability analysis supports a stronger Nash claim.

10.9 How PMP/FBSM baselines strengthen MADRL papers

A cyber MADRL paper is stronger when it is not compared only with random or static policies. PMP/FBSM or HODGE-style strategies can serve three roles.

1. **Baseline:** compare learned feedback policies with open-loop continuous-time strategies.
2. **Teacher:** generate state-action pairs from many initial states and pretrain the actor by behavior cloning.
3. **Validation:** evaluate whether learned actions approximately satisfy Hamiltonian stationarity in regions where the action space is continuous.

For example, if PMP gives $u_D^{\text{PMP}}(t)$ for an initial state x_0 , simulate small perturbations of x_0 , solve FBSM repeatedly, and train a policy $\pi_D(x, t)$ to imitate the resulting controls. Then fine-tune by RL/MADRL. This produces a more transparent bridge between differential-game theory and learning.

11. Neural ODE, PINN, and PIDL: What They Do Differently

11.1 Neural ODE as a dynamics learner

Neural ODE represents continuous dynamics by a neural network [14]:

$$\dot{x} = f_{\theta}(x, t). \quad (99)$$

For cyber research, the most useful version is often hybrid mechanistic-neural:

$$\dot{x} = f_{\text{known}}(x, u, t) + g_{\theta}(x, u, t). \quad (100)$$

The known term captures malware, APT, or rumor theory; the neural term captures unknown behavior such as attacker adaptation, social reinforcement, hidden vulnerabilities, or platform recommendation effects. Neural ODE learns dynamics; it still needs PMP, MPC, RL, or PINN/PIDL to solve control.

11.2 PINN/PIDL as equation-constrained learning

PINNs train networks to satisfy data and differential equation residuals [15]. Approximate

$$\hat{x}(t) = N_x(t), \quad \hat{u}(t) = N_u(t), \quad (101)$$

and define

$$r_x(t) = \dot{\hat{x}}(t) - f(\hat{x}(t), \hat{u}(t), t; \theta). \quad (102)$$

A basic loss is

$$\mathcal{L} = \mathcal{L}_{data} + \alpha_f \frac{1}{N_f} \sum_{j=1}^{N_f} \|r_x(t_j)\|^2 + \alpha_b \mathcal{L}_{boundary}. \quad (103)$$

PIDL is broader than PINN: it can include ODE/PDE residuals, PMP residuals, HJB/HJI residuals, conservation laws, parameter priors, monotonicity constraints, and cyber-specific constraints.

11.3 PMP-informed PINN

A true PMP-informed neural solver approximates state, costate, and control:

$$\hat{x}(t) = N_x(t), \quad \hat{\lambda}(t) = N_\lambda(t), \quad \hat{u}(t) = N_u(t). \quad (104)$$

The loss is

$$\mathcal{L}_{PMP} = \frac{1}{N_f} \sum_j \|\dot{\hat{x}} - H_\lambda\|^2 + \frac{1}{N_f} \sum_j \|\dot{\hat{\lambda}} + H_x\|^2 \quad (105)$$

$$+ \frac{1}{N_f} \sum_j \|H_u\|^2 + \mathcal{L}_{IC} + \mathcal{L}_{terminal}. \quad (106)$$

This is different from Level 1 DRL: the Hamiltonian and costate equations appear inside the training objective.

11.4 How Neural ODE is trained in a cyber setting

Suppose observations $x(t_i)$ are available. A Neural ODE predicts

$$\hat{x}(t_{i+1}) = \text{ODESolve}(f_\theta, \hat{x}(t_i), [t_i, t_{i+1}]). \quad (107)$$

The loss is

$$\mathcal{L}_{NODE} = \sum_i \|\hat{x}(t_i) - x^{obs}(t_i)\|^2. \quad (108)$$

For cyber models, a safer design is not to learn all of f from scratch. Instead use

$$f_\theta(x, u, t) = f_{mechanistic}(x, u, t; \theta_p) + g_\theta(x, u, t), \quad (109)$$

where $f_{mechanistic}$ encodes known propagation and g_θ learns missing mechanisms. This improves interpretability and reduces the amount of data required.

11.5 When PINN/PIDL is preferable to Neural ODE

Neural ODE primarily learns a trajectory-generating vector field from data. PINN/PIDL is preferable when one wants to enforce equations at collocation points, learn parameters under sparse data, impose conservation laws, or solve optimality systems. In short, Neural ODE is a dynamics learner; PINN/PIDL is an equation-constrained learning and solving framework. They can also be combined: Neural ODE learns unknown terms, while PINN/PIDL enforces physics and control residuals.

12. Hybrid Control and Mixed Continuous-Discrete Differential Games

12.1 Why hybrid actions are natural

In practice, a defender chooses both a mode and an intensity:

$$a_D^k = (m_D^k, v_D^k), \quad m_D^k \in \{\text{patch, monitor, isolate, deceive}\}, \quad v_D^k \in [0, 1]. \quad (110)$$

The attacker may choose

$$a_A^k = (m_A^k, v_A^k), \quad m_A^k \in \{\text{scan, exploit, lateral, stealth}\}, \quad v_A^k \in [0, 1]. \quad (111)$$

Between decision times,

$$\dot{x} = f_{m_A^k, m_D^k}(x, v_A^k, v_D^k, t), \quad t \in [t_k, t_{k+1}). \quad (112)$$

This is a hybrid differential game: continuous state dynamics, discrete strategic modes, and continuous resource intensities.

12.2 Solution routes

Route 1: mode enumeration plus PMP. Fix a mode sequence, solve continuous intensities by PMP/FBSM, and compare candidate sequences. This is interpretable but grows exponentially with the number of switching times.

Route 2: parameterized action RL. Use two policy heads [13]:

$$\pi(s) = (\pi_m(m|s), \pi_v(v|s, m)). \quad (113)$$

The first head selects a mode; the second outputs the intensity for that mode.

Route 3: hybrid-action MARL. Both attacker and defender use parameterized policies, and a centralized critic evaluates

$$Q(s, m_A, v_A, m_D, v_D). \quad (114)$$

This is appropriate for continuous-discrete cyber games.

Route 4: relaxed PINN/PIDL. Represent discrete modes with soft weights

$$\omega_m(t) = \text{softmax}(g_\theta(t)), \quad \sum_m \omega_m(t) = 1, \quad (115)$$

and use relaxed dynamics

$$\dot{x} = \sum_m \omega_m(t) f_m(x, v_m, t). \quad (116)$$

After training, select the dominant mode $m^*(t) = \arg \max_m \omega_m(t)$. This is not exact mixed-integer control, but it provides a differentiable approximation.

Route 5: impulse control and impulse games. Some actions, such as emergency isolation or forced password reset, are instantaneous. They can be modeled as

$$\dot{x}(t) = f(x, u, t), \quad t \neq \tau_j, \quad (117)$$

$$x(\tau_j^+) = G(x(\tau_j^-), a_j). \quad (118)$$

This connects with impulsive APT defense and outsourcing models [19, 20].

12.3 How to choose among hybrid solution routes

The five routes above have different purposes. If the main contribution is mathematical structure, use switching PMP or impulse games. If the main contribution is an operational adaptive defender, use hybrid-action RL or MARL. If the main contribution is data-efficient learning under sparse observations, use PINN/PIDL for parameter learning and then use RL/MARL for the discrete strategic layer. A balanced cyber paper can combine them: PINN/PIDL estimates unknown propagation parameters, PMP/FBSM provides a continuous-control benchmark, and MARL learns hybrid feedback policies under adaptive attackers.

12.4 Example hybrid action mapping

For defender mode m_D and intensity v_D , define

$$\rho_k = \rho_0 + v_D \rho_{\max} \mathbf{1}\{m_D = \text{patch}\}, \quad (119)$$

$$\mu_k = \mu_0 + v_D \mu_{\max} \mathbf{1}\{m_D = \text{monitor}\}, \quad (120)$$

$$\iota_k = v_D \iota_{\max} \mathbf{1}\{m_D = \text{isolate}\}, \quad (121)$$

$$\eta_k = v_D \eta_{\max} \mathbf{1}\{m_D = \text{deceive}\}. \quad (122)$$

This map makes the model transparent. It also makes it clear which part is learned: the policy learns m_D and v_D , while the ODE defines how these choices affect cyber dynamics.

13. Detailed Case Studies

13.1 Case 0: HODGE-style hybrid propaganda war game

This case is useful because it contains several ingredients that should also appear in future MADRL extensions: two strategic parties, degree-based spreading dynamics, continuous controls, impulsive controls, and approximate Nash evaluation [18]. A simplified implementation plan is:

1. **State:** use a degree-based summary such as average proportions of ignorant, propaganda-spreading, and counter-propaganda-spreading users for each party. If the full degree distribution is retained, the RL state can include $[P(1), \dots, P(K), C(1), \dots, C(K)]$ for each OSN.
2. **Actions:** each party selects a continuous propaganda investment level and an optional impulsive counter-propaganda intervention. For MADRL, this can be encoded as a hybrid action (m_i^k, u_i^k, v_i^k) , where m_i^k decides whether an impulse is applied.
3. **Transition:** if $m_i^k = 1$, apply the jump map to shift some propaganda spreaders or ignorant users toward counter-propaganda states; then integrate the degree-based ODE until the next decision epoch.
4. **Reward:** combine influence benefit, counter-influence benefit, continuous investment cost, impulse cost, and terminal state benefit.
5. **Solver comparison:** compare static investment, random hybrid investment, HODGE-style iterative strategies, independent PPO, and CTDE-based MADRL.
6. **Evaluation:** report payoff, cost-effectiveness, timing of impulses, lower-bound sensitivity, and robustness to different OSN degree distributions.

The main message for students is that HODGE is not merely a test case. It is a template for translating hybrid differential games into Markov games without losing the meaning of continuous and impulsive controls.

13.2 Case A: malware defense by DDQN

1. Start from the malware ODE and choose Δt .
2. Convert state $x(t_k)$ to $s_k = [S_k, I_k, R_k, b_k]$.
3. Define discrete defense actions.
4. Implement `env.step( $a_k$ )` by mapping a_k to ODE parameters and integrating the ODE.
5. Define reward as negative infection and action cost.
6. Train DDQN using replay buffer and target network.
7. Compare with no defense, static patching, threshold defense, and FBSM continuous-control baseline.

This is Level 1 coupling: the continuous cyber model defines the MDP; DDQN solves the feedback policy. DDQN-based malware suppression follows this general logic [21, 24, 25].

13.3 Case B: APT defense by PMP-informed PINN

1. Build the APT model with $x = [V, C, P, z]$.
2. Define the defender cost with compromise loss, hardening cost, repair cost, isolation cost, and deception cost.
3. Derive $H, \dot{x} = H_\lambda, \dot{\lambda} = -H_x$, and $H_u = 0$.
4. Build three neural networks $N_x(t)$, $N_\lambda(t)$, and $N_u(t)$.
5. Train using state residual, costate residual, stationarity residual, initial condition, terminal condition, and control-bound penalties.
6. Compare with FBSM and direct control PINN.

This is Level 3 coupling because the PMP system appears directly in the loss.

13.4 Case C: misinformation defense by sequential PINN plus optimal control

1. Use sparse time-series data for $M(t)$ and possibly $C(t)$.
2. Train an inverse PINN to estimate $\beta_M, \beta_C, \alpha, \delta, \delta_C$.
3. Fix learned parameters and solve optimal correction control $u_f(t)$ by FBSM or PMP-informed PINN.
4. Evaluate cumulative misinformation, peak misinformation, correction cost, and robustness to parameter uncertainty.

This sequential pipeline is common in PINN-based rumor and negative-information studies [28, 29, 30].

13.5 Case D: hybrid APT game by PINN/PIDL plus MARL

This case shows a complete pipeline that avoids forcing one method to do everything.

1. Use sparse observations of compromised assets $C(t)$ and possibly deception alerts to estimate parameters β, χ, γ, ξ with inverse PINN/PIDL.
2. Fix or sample from the learned parameter distribution and define the hybrid Markov game with defender actions (m_D, v_D) and attacker actions (m_A, v_A) .
3. Train a defender policy against scripted attackers using PPO or hybrid-action actor-critic.
4. Train an adaptive attacker against the defender.
5. Start self-play with CTDE critics.
6. Compare against static deception, static patching, threshold rules, FBSM continuous-control baselines, and no-defense baselines.

This design is methodologically clean: PINN/PIDL handles parameter learning, PMP/FBSM gives theory and baseline controls, and MARL handles discrete-continuous strategic adaptation.

14. How to Report Results

Reward curves alone are not enough. Report cyber metrics:

- cumulative compromise or misinformation $\int_0^T C(t)dt$ or $\int_0^T M(t)dt$;
- peak compromise or peak misinformation;
- final compromised or misinformation level;
- total defense cost;
- containment time;
- action distribution for RL/MARL;
- learned parameter error for PINN/PIDL;
- robustness under parameter perturbation;
- computation time for FBSM, DRL inference, and PINN training.

15. Implementation Cookbooks for Cyber Control and Games

This section turns the previous modeling discussion into concrete implementation recipes. The goal is to make clear what a researcher or student actually does in code. The recipes are intentionally modular. The same malware, APT, or misinformation model can be solved by FBSM, DQN, PPO, SAC, MARL, Neural ODE, or PINN/PIDL, but the computational objects are different.

15.1 Cookbook A: FBSM for continuous malware control

Consider the malware model

$$\dot{S} = -\beta SI - u_p S + \omega R, \quad (123)$$

$$\dot{I} = \beta SI - \gamma I - u_c I, \quad (124)$$

$$\dot{R} = \gamma I + u_p S + u_c I - \omega R, \quad (125)$$

with objective

$$J = \int_0^T \left[A_I I + \frac{B_p}{2} u_p^2 + \frac{B_c}{2} u_c^2 \right] dt + A_T I(T). \quad (126)$$

The Hamiltonian is

$$\begin{aligned} H = & A_I I + \frac{B_p}{2} u_p^2 + \frac{B_c}{2} u_c^2 + \lambda_S (-\beta SI - u_p S + \omega R) \\ & + \lambda_I (\beta SI - \gamma I - u_c I) + \lambda_R (\gamma I + u_p S + u_c I - \omega R). \end{aligned} \quad (127)$$

The stationarity conditions give candidate controls

$$\tilde{u}_p(t) = \frac{(\lambda_S - \lambda_R)S}{B_p}, \quad (128)$$

$$\tilde{u}_c(t) = \frac{(\lambda_I - \lambda_R)I}{B_c}. \quad (129)$$

With bounds $0 \leq u_p \leq u_{p,\max}$ and $0 \leq u_c \leq u_{c,\max}$,

$$u_p^*(t) = \text{clip} \left(\frac{(\lambda_S - \lambda_R)S}{B_p}, 0, u_{p,\max} \right), \quad (130)$$

$$u_c^*(t) = \text{clip} \left(\frac{(\lambda_I - \lambda_R)I}{B_c}, 0, u_{c,\max} \right). \quad (131)$$

FBSM implementation steps.

1. Create a time grid $0 = t_0 < t_1 < \dots < t_N = T$.
2. Initialize controls $u_p^{(0)}(t_i), u_c^{(0)}(t_i)$, often zeros or small constants.
3. Forward step: solve the state ODE from $t = 0$ to T using the current controls.
4. Backward step: solve the costate ODE from T to 0 using terminal conditions such as $\lambda_I(T) = A_T$.
5. Control update: compute $\tilde{u}_p, \tilde{u}_c$ from stationarity and project them into admissible intervals.
6. Relaxation: update $u^{(q+1)} = \alpha \tilde{u} + (1 - \alpha)u^{(q)}$, with $\alpha \in [0.1, 0.7]$.
7. Stop when $\|u^{(q+1)} - u^{(q)}\|, \|x^{(q+1)} - x^{(q)}\|$, and $\|\lambda^{(q+1)} - \lambda^{(q)}\|$ are small.

FBSM produces an *open-loop* control curve $u^*(t)$ for a given initial condition and parameter set. It is usually fast and interpretable for low-dimensional smooth models. Its weakness is that if the state unexpectedly changes, the precomputed open-loop control is not automatically adaptive.

15.2 Cookbook B: DQN/DDQN for discrete cyber defense

To use DQN, convert the continuous model into an MDP. Let

$$s_k = [S(t_k), I(t_k), R(t_k)] \quad (132)$$

and define a discrete action set

$$\mathcal{A}_D = \{\text{no defense, patch, clean, patch+clean}\}. \quad (133)$$

Each action maps to a piecewise-constant control over $[t_k, t_{k+1}]$. For example,

Action	u_p	u_c
no defense	0	0
patch	$u_{p,H}$	0
clean	0	$u_{c,H}$
patch+clean	$u_{p,M}$	$u_{c,M}$

The environment integrates the ODE for one interval and returns

$$s_{k+1} = \text{ODESolve}(f, s_k, a_k, \Delta t). \quad (134)$$

The reward is the negative interval cost:

$$r_k = - \int_{t_k}^{t_{k+1}} \left[A_I I(t) + \frac{B_p}{2} u_p^2 + \frac{B_c}{2} u_c^2 \right] dt. \quad (135)$$

In code, this integral can be approximated by the trapezoidal rule or by averaging values returned by the ODE solver.

DQN/DDQN implementation steps.

1. Define a simulator function `step(state, action)` that maps the action to controls, solves the ODE, computes reward, and returns next state.
2. Define a Q-network $Q_\theta(s, a)$ with input s and output one Q-value per discrete action.
3. Use ϵ -greedy exploration: choose a random action with probability ϵ , otherwise choose $\arg \max_a Q_\theta(s, a)$.
4. Store transitions $(s_k, a_k, r_k, s_{k+1}, d_k)$ in a replay buffer.
5. Sample mini-batches from the buffer. For DQN, use

$$y = r + \gamma_d \max_{a'} Q_{\bar{\theta}}(s', a').$$

6. For Double DQN, select the next action using the online network but evaluate it using the target network:

$$y = r + \gamma_d Q_{\bar{\theta}} \left(s', \arg \max_{a'} Q_{\theta}(s', a') \right).$$

7. Minimize $(y - Q_\theta(s, a))^2$ and periodically update the target network.

DQN is suitable when defense actions are naturally discrete. It is less suitable when the main decision is a continuous intensity, such as patching rate or deception strength. In that case, use PPO, DDPG, TD3, or SAC.

15.3 Cookbook C: PPO, DDPG, and SAC for continuous control

For continuous defense intensities, define

$$a_k = [u_p^k, u_c^k] \in [0, u_{p,\max}] \times [0, u_{c,\max}]. \quad (136)$$

The action remains constant on $[t_k, t_{k+1}]$. PPO uses a stochastic policy, for example

$$\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s)), \quad (137)$$

followed by squashing or clipping to the control bounds. DDPG and SAC use actor-critic structures for continuous actions [7, 9]. SAC is often more stable because the entropy term encourages exploration.

Algorithm	Suitable cyber action	Practical note
PPO	discrete or continuous actions	robust default; good for first experiments
DDPG	deterministic continuous intensities	sensitive to exploration noise and tuning
SAC	stochastic continuous intensities	strong choice when reward landscape is complex
TD3	continuous intensities	improves DDPG using twin critics and delayed actor updates

For cyber control, the policy should be evaluated not only by reward but also by security metrics: cumulative compromise, peak compromise, final compromise, control cost, and containment time.

15.4 Cookbook D: MARL and CTDE for attacker-defender games

For an attacker-defender game, define a Markov game state

$$s_k = [V_k, C_k, P_k, z_k], \quad (138)$$

defender action a_D^k , attacker action a_A^k , and transition

$$s_{k+1} = F_{\Delta t}(s_k, a_D^k, a_A^k). \quad (139)$$

The transition is implemented by an ODE solver. Rewards may be zero-sum,

$$r_A^k = -r_D^k, \quad (140)$$

or general-sum, where the attacker's reward and defender's cost are related but not exact negatives.

CTDE implementation steps.

1. Actors: define $\pi_D(a_D|o_D)$ and $\pi_A(a_A|o_A)$, where observations can be the full state or partial observations.
2. Centralized critics: define $Q_D(s, a_D, a_A)$ and $Q_A(s, a_D, a_A)$ during training.
3. Rollout: at each step, both agents choose actions, the ODE environment advances one interval, and rewards are stored.
4. Update critics using joint state-action information.
5. Update each actor using its own objective and the centralized critic.
6. During execution, remove centralized information and let each actor use its local observation.

This is more realistic than single-agent RL because attackers adapt. However, training can be unstable because each agent changes the other agent's environment. Stabilization techniques include self-play with policy snapshots, opponent pools, slower attacker updates, reward normalization, and curriculum training from simple attackers to adaptive attackers.

15.5 Cookbook E: Hybrid continuous-discrete action games

Many cyber decisions are hybrid:

$$a_D^k = (m_D^k, v_D^k), \quad m_D^k \in \{\text{patch, monitor, isolate, deceive}\}, \quad v_D^k \in [0, 1]. \quad (141)$$

Here m_D^k is the discrete mode and v_D^k is its continuous intensity. A two-head policy is natural:

$$\pi_D^{mode}(m|s) = \text{softmax}(g_\theta(s)), \quad (142)$$

$$v_D = v_{\max} \text{sigmoid}(h_\theta(s, m)). \quad (143)$$

The mode head chooses what type of defense is used; the intensity head chooses how strongly it is used. This structure matches parameterized-action RL [13]. For MARL, the attacker can have a similar two-head action structure.

Implementation advice. Start with purely discrete actions to debug the environment. Then add continuous intensities. Finally add adaptive attackers. Many failed DRL projects fail because the environment, reward, and action mapping are not tested with simple baselines first.

15.6 Reward shaping without changing the research objective

A reward should correspond to the original objective, but shaping can improve learning. For example,

$$r_k = -\Delta t [A_C C_{k+1} + B_u \|u_k\|^2] - \eta_{peak} \max(0, C_{k+1} - C_{safe}). \quad (144)$$

The first part is the discretized optimal-control cost. The last term penalizes unsafe peaks. If reward shaping is used, the paper should report both the shaped training reward and the original scientific metrics.

15.7 Testing the environment before training

Before running DRL, test the environment with deterministic action sequences:

1. no defense;
2. always patch;
3. always clean;
4. periodic patching;
5. threshold policy, e.g., clean if $I > 0.2$;
6. random actions.

The DRL policy should outperform or at least match these baselines. If it does not, the issue may be reward scaling, action mapping, exploration, or simulator bugs.

16. Extra Implementation Guidance: From Equations to Experiments

The previous cookbooks describe the main algorithms. This section adds lower-level implementation details that are often omitted in papers but are important for producing convincing, reproducible results.

16.1 A minimal ODE-RL environment: objects and data flow

For an ODE-based cyber-defense environment, each environment step should perform four tasks: decode the action, integrate the ODE, compute the reward, and return the next observation. A clean implementation keeps these tasks separate.

Recommended environment functions.

1. `reset()`: sample initial state x_0 , set $k = 0$, return observation $s_0 = \psi(x_0)$.
2. `decode_action(a)`: convert the RL action into ODE controls u_D^k , and possibly u_A^k .
3. `rhs(t, x, u, theta)`: compute $f(x, u, \theta, t)$.
4. `integrate(x, u)`: compute $x_{k+1} = \text{ODESolve}(f, x_k, u, [t_k, t_{k+1}])$.
5. `reward(x, u, x_next)`: approximate $-\int_{t_k}^{t_{k+1}} L(x(t), u, t) dt$.
6. `observe(x)`: return the part of the state visible to the learner.

This design is better than embedding all logic into one large function. It lets the researcher test whether the ODE is correct before training RL, whether the reward has the intended sign and scale, and whether a learned policy is exploiting numerical artifacts.

16.2 What the RL state should include

In a pure mathematical model, the state is simply $x(t)$. In a cyber implementation, the observation may include derived quantities:

$$s_k = [x(t_k), \Delta x_k, b_k, h_k, \tau_k]. \quad (145)$$

Here $\Delta x_k = x_k - x_{k-1}$ is a trend feature, b_k is remaining budget, h_k is a short action history, and $\tau_k = k/K$ is normalized time. These features can help a DRL agent learn time-dependent strategies. For example, a defender may patch aggressively early, deceive during fast spread, and monitor when $C(t)$ becomes small. If s_k only contains C_k , the agent may not know whether compromise is increasing or decreasing.

However, adding too many features can make interpretation harder. A good research protocol is to start from

$$s_k = [V_k, C_k, P_k, z_k] \quad (146)$$

and then report an ablation study showing whether trend, budget, or time features improve performance.

16.3 Reward scale and sign checks

Before training, evaluate the reward of simple policies. For example, compare no defense, always patch, always isolate, and rule-based defense. If no defense receives a better average return than reasonable

defensive policies, the reward is probably incorrectly signed or badly scaled. A typical defender reward is

$$r_D^k = -\Delta t \left(w_C C_{k+1} + c_p u_{p,k}^2 + c_m u_{m,k}^2 + c_i u_{i,k}^2 + c_d u_{d,k}^2 \right). \quad (147)$$

This reward should usually be negative. Less negative means better. If one wants positive rewards, define a reference loss $L_{\max}$ and use

$$r_D^k = L_{\max} - \left(w_C C_{k+1} + c_p u_{p,k}^2 + c_m u_{m,k}^2 + c_i u_{i,k}^2 + c_d u_{d,k}^2 \right). \quad (148)$$

The transformation should not change the policy preference if it is applied consistently. The most important point is that the reward should match the original control objective.

16.4 ODE solver choice in RL experiments

In RL, the ODE solver is part of the environment transition. A high-accuracy solver reduces numerical error but can slow training. A practical choice is:

- use fixed-step RK4 for fast and reproducible training;
- use an adaptive solver, such as Runge-Kutta 45, to validate final policies;
- clip or project the state after each step to preserve physical constraints;
- test whether smaller Δt changes the ranking of policies.

For compartment models, projection is often necessary because numerical integration can produce tiny negative values. A simple projection is

$$\tilde{x}_i = \max(x_i, 0), \quad x_i^{\text{proj}} = \frac{\tilde{x}_i}{\sum_j \tilde{x}_j}. \quad (149)$$

This should be used carefully. If projection is frequently large, the solver step size is too coarse or the ODE is unstable.

16.5 How to document an RL experiment

A cyber-control DRL paper should report more than final reward. At minimum, report:

1. the continuous-time model and all parameters;
2. the decision interval Δt and total horizon T ;
3. the state/observation vector s_k ;
4. the action set or continuous action bounds;
5. the action-to-control map Γ ;
6. the reward formula and its relation to the continuous-time cost;
7. the ODE solver and state projection rule;
8. the training algorithm, network size, learning rate, batch size, replay buffer or rollout length;
9. baselines and random seeds;

10. trajectory plots, not only reward curves.

This list makes the work reproducible and helps reviewers see that the learning method is solving the stated control or game problem.

16.6 Specific implementation choices for Section 10 cookbooks

The cookbooks in Section 10 can be implemented in a staged way.

Stage 1: deterministic simulator. Implement the ODE and run all static controls. Plot $C(t)$, $V(t)$, and accumulated cost. At this stage no neural network is needed.

Stage 2: discrete-action MDP. Map each action to a fixed control vector. For example,

$$\Gamma(0) = (u_p, u_m, u_i, u_d) = (0.0, 0.1, 0.0, 0.0), \quad \text{monitor}, \quad (150)$$

$$\Gamma(1) = (0.8, 0.0, 0.0, 0.0), \quad \text{patch}, \quad (151)$$

$$\Gamma(2) = (0.0, 0.0, 0.8, 0.0), \quad \text{isolate}, \quad (152)$$

$$\Gamma(3) = (0.0, 0.0, 0.0, 0.8), \quad \text{deceive}. \quad (153)$$

Train DQN or DDQN on this MDP. This is the easiest way to verify the whole learning pipeline.

Stage 3: continuous-action control. Let the actor output $u_k \in [0, 1]^m$. Use a squashing function such as

$$u_k = \frac{1}{2} (1 + \tanh(g_\phi(s_k))), \quad (154)$$

where g_ϕ is the actor network. PPO, SAC, or DDPG can be used here.

Stage 4: hybrid action. Let the policy output both a mode and an intensity:

$$p_m = \text{softmax}(g_\phi^m(s_k)), \quad v = \text{sigmoid}(g_\phi^v(s_k, m)). \quad (155)$$

The selected action is $a_k = (m_k, v_k)$. In practice, the discrete mode can be sampled from p_m , while the intensity is produced by a mode-conditioned head. This is useful for cyber defense because real interventions are often of the form “choose a defense mode and choose how strongly to apply it.”

16.7 Connecting PMP more tightly to the RL implementation

If one wants the RL part to be more clearly connected to PMP, there are several concrete options.

1. **PMP baseline:** compute an FBSM open-loop control and compare DRL feedback policies against it.
2. **Imitation warm start:** train a policy network to imitate FBSM controls at sampled states, then fine-tune with RL.
3. **Reward shaping:** add a penalty when the action deviates from the PMP stationarity condition, for example

$$r_k^{\text{shape}} = r_k - \eta_H \|H_u(x_k, u_k, \hat{\lambda}_k)\|^2. \quad (156)$$

Here $\hat{\lambda}_k$ may come from an FBSM solution or a learned costate network.

4. **Critic initialization:** use approximate value or costate information from PMP/HJB to initialize the critic.
5. **Policy validation:** after training, evaluate whether learned controls approximately satisfy stationarity along the generated trajectory.

These choices make the relationship between PMP and learning explicit. Without one of these links, a paper should honestly describe the method as simulator-based DRL inspired by, but not directly solving, the PMP optimality system.

17. Practical Python Implementation Roadmap

This section explains how the Python code in Appendix I should be used. The code is intentionally written as skeleton code rather than a complete software package. It is designed to show the exact mapping from equations to implementation.

1. **Implement the ODE simulator first.** Before training any RL agent, implement a deterministic function that maps (x_k, a_k) to x_{k+1} using Euler or RK4. Check positivity, conservation, and whether the state reacts to actions in the expected direction.
2. **Wrap the simulator as an environment.** The environment should have `reset()` and `step(action)`. The `step` function should decode the action, integrate the ODE, compute reward, and return the next observation.
3. **Train a simple DQN before a complex DRL/MARL method.** DQN is not always the final method, but it is the best debugging tool for discrete defense actions. If DQN cannot learn against a scripted attacker, MAPPO or SAC will usually not fix the modeling error.
4. **Only then add continuous or hybrid actions.** Continuous intensities can be handled with DDPG/SAC/PPO. Hybrid actions require both a discrete mode and a continuous intensity. Appendix F gives the mathematical background.
5. **For attacker-defender games, start with scripted opponents.** Train the defender against fixed attackers, then train the attacker against fixed defenders, and only then move to self-play or CTDE.
6. **Compare against PMP/FBSM carefully.** FBSM gives an open-loop continuous-control benchmark. RL gives a feedback policy for a discretized environment. They should be compared using the same cost functional, initial conditions, action limits, and horizon.

Implementation warning. Most failed cyber DRL experiments fail because the environment is wrong, not because the neural network is too small. Always plot uncontrolled trajectories, static-action trajectories, and reward components before training.

18. Main Takeaways

PMP and FBSM provide theoretical structure, optimality conditions, and interpretable continuous-control baselines. RL and DRL learn feedback policies for MDPs induced by continuous-time cyber models.

MARL extends this to adaptive attackers and defenders through Markov games. Neural ODE learns unknown continuous-time dynamics. PINN and PIDL embed equations, constraints, and possibly PMP or HJI residuals into neural training. Hybrid control is especially important in cybersecurity because real interventions combine discrete modes and continuous intensities.

19. Python Implementation Map and Code Organization

The accompanying code is intentionally organized around the same methodological structure as this note. The mathematical model is separated from the solver so that FBSM, DDQN, MADRL, PINN, and PIDL can be compared on related cyber dynamics rather than on unrelated toy problems.

Python file	What it demonstrates
<code>cyber_dynamics.py</code>	ODE right-hand sides, RK4 integration, and state projection for compartment models.
<code>cyber_hybrid_env.py</code>	The central sampled-data hybrid environment: observe, decode action, apply jump map, integrate ODE, compute rewards.
<code>fbsm_malware_baseline.py</code>	Classical PMP/FBSM baseline for continuous malware control.
<code>ddqn_cyber_defense.py</code>	Discrete-action DDQN defender against a scripted attacker.
<code>madrl_ctde_hybrid_game.py</code>	A compact CTDE/MADRL attacker-defender Markov game.
<code>inverse_pinn_sir_malware.py</code>	Inverse PINN for learning propagation parameters.
<code>pidl_unknown_mechanism.py</code>	PIDL model with known dynamics plus a learned missing mechanism.
<code>control_pinn_malware.py</code> and <code>pmp_informed_pinn_malware.py</code>	Direct control PINN and PMP-informed PINN baselines.

Recommended implementation sequence. First run the ODE environment and FBSM baseline. Then train DDQN against a scripted attacker. Only after these two parts are debugged should one move to simultaneous attacker-defender MADRL. PINN/PIDL scripts should be used to learn parameters or missing mechanisms before claiming data-driven control.

A. Deep Learning Background: Networks, Activations, and Optimization

A feedforward neural network represents a function by composing affine maps and nonlinear activations:

$$h^{(0)} = s, \quad (157)$$

$$h^{(\ell+1)} = \sigma \left(W^{(\ell)} h^{(\ell)} + b^{(\ell)} \right), \quad (158)$$

$$y = W^{(L)} h^{(L)} + b^{(L)}. \quad (159)$$

In cyber-control applications, the input s may be an observation such as $[V, C, P, z]$, and the output may be Q-values, action probabilities, continuous controls, or value estimates. The parameters are all matrices and vectors

$$\Theta = \{W^{(\ell)}, b^{(\ell)}\}_{\ell=0}^L. \quad (160)$$

Training means minimizing a loss function by gradient-based optimization:

$$\Theta \leftarrow \Theta - \alpha \nabla_{\Theta} \mathcal{L}(\Theta), \quad (161)$$

where α is the learning rate. In practice, Adam is often used before more specialized optimizers.

A.1 Sigmoid, tanh, and softmax

The sigmoid function

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}} \quad (162)$$

maps a real number to $(0, 1)$, which is useful for bounded controls such as patching intensity. If a control has bounds $u \in [u_{\min}, u_{\max}]$, one can set

$$u = u_{\min} + (u_{\max} - u_{\min}) \text{sigmoid}(z). \quad (163)$$

The softmax function maps logits ℓ_i to probabilities:

$$p_i = \frac{e^{\ell_i/\tau}}{\sum_{j=1}^m e^{\ell_j/\tau}}, \quad i = 1, \dots, m. \quad (164)$$

Here $\tau > 0$ is a temperature. Large τ produces smoother probabilities; small τ makes the distribution close to one-hot. Softmax is used in policy networks for discrete actions such as patch, monitor, isolate, and deceive.

A.2 Why scaling matters

Neural networks train more reliably when inputs have similar scales. For example, if $C \in [0, 1]$ but budget is measured in thousands, the network may overemphasize budget. Normalize features before training:

$$\tilde{s}_i = \frac{s_i - \mu_i}{\sigma_i + 10^{-8}}. \quad (165)$$

For compartment models, states are often already in $[0, 1]$, which is convenient.

B. ODE Solver Background for Cyber Control

ODE solvers approximate the solution of

$$\dot{x} = f(x, u, t), \quad x(t_k) = x_k. \quad (166)$$

In an ODE-RL environment, the solver defines the transition from x_k to x_{k+1} .

B.1 Euler and RK4

The forward Euler method is

$$x_{k+1} = x_k + \Delta t f(x_k, u_k, t_k). \quad (167)$$

It is simple but may be inaccurate or unstable for large Δt . A more reliable fixed-step method is RK4:

$$k_1 = f(x_k, u_k, t_k), \quad (168)$$

$$k_2 = f\left(x_k + \frac{\Delta t}{2} k_1, u_k, t_k + \frac{\Delta t}{2}\right), \quad (169)$$

$$k_3 = f\left(x_k + \frac{\Delta t}{2} k_2, u_k, t_k + \frac{\Delta t}{2}\right), \quad (170)$$

$$k_4 = f(x_k + \Delta t k_3, u_k, t_k + \Delta t), \quad (171)$$

$$x_{k+1} = x_k + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4). \quad (172)$$

RK4 is often a good default for repeated RL training because it is deterministic and easy to reproduce.

B.2 Adaptive solvers

Adaptive solvers select internal step sizes automatically. They are useful for validation but can be slower. When using adaptive solvers in RL, keep the outer decision interval Δt fixed, even if the solver uses smaller internal steps. Otherwise the action timing changes across episodes.

B.3 Conservation and positivity

Compartment models often require $S + I + R = 1$ and nonnegative states. Numerical solvers may slightly violate these constraints. Use either smaller steps, constrained parameterizations, or projection. Projection should be reported because it changes the environment transition.

C. MDP, Bellman Equation, and Q-Learning Background

An MDP consists of states s , actions a , transition probabilities $P(s'|s, a)$, rewards $r(s, a, s')$, and discount factor γ_d . The goal is to find a policy $\pi(a|s)$ that maximizes

$$G_k = \sum_{j=0}^{K-k} \gamma_d^j r_{k+j}. \quad (173)$$

The optimal Q-function satisfies the Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma_d \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (174)$$

Classical Q-learning updates a table by

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma_d \max_{a'} Q(s', a') - Q(s, a) \right]. \quad (175)$$

DQN replaces the table by a neural network $Q_\theta(s, a)$ [5].

C.1 DQN and DDQN details

DQN minimizes

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{B}} \left[(y - Q_\theta(s,a))^2 \right], \quad (176)$$

where $\mathcal{B}$ is a replay buffer and

$$y = r + \gamma_d \max_{a'} Q_{\theta^-}(s', a') \quad (177)$$

uses a target network θ^- . Double DQN reduces overestimation by decoupling action selection and evaluation:

$$y = r + \gamma_d Q_{\theta^-} \left(s', \arg \max_{a'} Q_\theta(s', a') \right). \quad (178)$$

In cyber defense, DQN is suitable when actions are discrete, for example no-op, patch, isolate, and deceive.

C.2 Practical DQN hyperparameters

A reasonable first configuration is: two hidden layers with 64 or 128 units, ReLU activations, Adam optimizer, learning rate 10^{-4} to 10^{-3} , replay buffer size 10^4 to 10^5 , batch size 64 or 128, and target-network update every 500 to 2000 environment steps. The exploration rate ϵ can decay from 1.0 to 0.05. These are starting points, not universal settings.

D. PPO and Continuous-Action Actor-Critic Background

PPO trains a stochastic policy and a value function. The policy gradient is stabilized by clipping the probability ratio

$$r_t(\phi) = \frac{\pi_\phi(a_t|s_t)}{\pi_{\phi_{old}}(a_t|s_t)}. \quad (179)$$

The clipped objective is

$$\mathcal{L}^{\text{PPO}}(\phi) = \mathbb{E} \left[\min \left(r_t(\phi) \hat{A}_t, \text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (180)$$

where $\hat{A}_t$ is an advantage estimate [8]. PPO is easy to use for both discrete and continuous actions.

D.1 DDPG and SAC

DDPG learns a deterministic actor $a = \mu_\phi(s)$ and a critic $Q_\theta(s, a)$ [7]. It is natural for continuous cyber controls but can be sensitive to exploration and hyperparameters. SAC learns a stochastic policy and maximizes both return and entropy:

$$J(\pi) = \mathbb{E} \left[\sum_k \gamma_d^k (r_k + \alpha \mathcal{H}(\pi(\cdot|s_k))) \right]. \quad (181)$$

The entropy term encourages exploration, which is useful when the best defense strategy changes over time [9].

E. Markov Game and MARL Background

A two-player Markov game has state s , defender action a_D , attacker action a_A , transition $P(s'|s, a_D, a_A)$, and rewards r_D, r_A [10]. In a zero-sum game,

$$r_A = -r_D. \quad (182)$$

In a general-sum game, the attacker and defender have different objectives.

E.1 Independent learning and nonstationarity

Independent learners treat other agents as part of the environment. This is simple but creates nonstationarity because the opponent's policy changes during training. In cyber games, this can appear as unstable training curves or policies that only work against one attacker.

E.2 Centralized training, decentralized execution

CTDE addresses nonstationarity by using a centralized critic during training:

$$Q_D(s, a_D, a_A), \quad Q_A(s, a_D, a_A). \quad (183)$$

At execution time, each actor uses only its own observation. This fits cyber defense: training may use full simulator information, but deployment may only use defender observations.

F. Hybrid Actions, Softmax Relaxation, and Action Masks

Hybrid cyber actions combine a discrete mode and a continuous intensity:

$$a = (m, v), \quad m \in \{1, \dots, M\}, \quad v \in [0, 1]. \quad (184)$$

A policy can output mode probabilities $p = \text{softmax}(g_m(s))$ and mode-conditioned intensities $v_m = \text{sigmoid}(g_v(s, m))$. The executed action uses one mode m and its corresponding intensity v_m .

F.1 Action masks

Some actions may be infeasible. For example, isolate may be unavailable when budget is too low. Let $M_i(s) \in \{0, 1\}$ be an action mask. A masked softmax can be written as

$$p_i = \frac{M_i(s)e^{\ell_i}}{\sum_j M_j(s)e^{\ell_j}}. \quad (185)$$

This prevents the policy from selecting impossible actions. The mask should represent real operational constraints, not arbitrary tuning.

F.2 Differentiable relaxation

For differentiable training, one can use a soft mixture of modes:

$$u(s) = \sum_{m=1}^M p_m(s) u_m(s). \quad (186)$$

This is not the same as executing a single discrete mode, but it is useful in PINN/PIDL or differentiable simulation. If the final system requires discrete actions, validate by converting the soft policy to hard actions.

G. Appendix: Game-Theoretic Background for MARL and MADRL

G.1 Normal-form game, dynamic game, differential game, and Markov game

A normal-form game describes one-shot interaction: each player chooses an action once and receives a payoff. A dynamic game describes repeated or time-evolving interaction. A differential game is a dynamic game in which the state follows a differential equation. A Markov game is a discrete-time feedback version in which the next state depends on the current state and joint actions.

Object	Mathematical form	Learning interpretation
Normal-form game	payoff $R_i(a_1, \dots, a_N)$	useful for local stage-game analysis
Differential game	$\dot{x} = f(x, u_1, \dots, u_N), J_i = \int L_i dt$	natural for cyber propagation and resource allocation
Markov game	$s_{k+1} \sim P(\cdot s_k, a_1^k, \dots, a_N^k)$	the standard environment model for MARL/MADRL
Hybrid Markov game	jump map plus ODE integration between decisions	suitable for patching, incident response, takedown, and propaganda interventions

G.2 Zero-sum and general-sum learning objectives

In a zero-sum game, the defender may solve

$$\max_{\pi_D} \min_{\pi_A} V_D^{\pi_D, \pi_A}(s), \quad (187)$$

or equivalently the attacker maximizes $-V_D$. In a general-sum game, each player has a separate objective:

$$\pi_i^* \in \arg \max_{\pi_i} V_i^{\pi_i, \pi_{-i}^*}(s), \quad i = 1, \dots, N. \quad (188)$$

Many cyber games are closer to general-sum than pure zero-sum because each side has different operational costs, constraints, and benefits. This is why MADRL papers should state clearly whether they train with one shared zero-sum reward, separate rewards, or shaped rewards.

G.3 Open-loop, feedback, and sampled-data interpretations

Open-loop PMP strategies are functions of time, $u_i(t)$. Feedback strategies are functions of state, $u_i(t, x)$. Sampled-data strategies sit between them: players observe the state at discrete times and then apply a control over the next interval. RL/MADRL most naturally implements sampled-data feedback:

$$a_i^k \sim \pi_i(\cdot | s_k), \quad u_i(t) = \mu_i(a_i^k), \quad t \in [t_k, t_{k+1}). \quad (189)$$

This is often more realistic in cyber settings than ideal continuous feedback because defenders review alerts, budgets, and threat intelligence at decision intervals rather than continuously measuring the entire network state.

G.4 MADDPG-style centralized critic

For continuous or hybrid actions, a centralized critic can be written as

$$Q_i(s, a_1, \dots, a_N; \phi_i). \quad (190)$$

The target is

$$y_i = r_i + \gamma Q_i^-(s', a'_1, \dots, a'_N), \quad a'_j \sim \pi_j^-(\cdot | o'_j), \quad (191)$$

and the critic minimizes $\mathbb{E}[(Q_i - y_i)^2]$. The actor of player i is updated to increase Q_i while other actions are treated as fixed samples. This is useful when attacker and defender actions are continuous intensities such as attack effort, deception level, or patching intensity.

G.5 MAPPO-style centralized value function

For discrete or mixed actions, MAPPO often uses a centralized value function $V_i(s)$ and decentralized policies $\pi_i(a_i | o_i)$. The clipped PPO objective for each agent is based on

$$\rho_i = \frac{\pi_i(a_i^k | o_i^k)}{\pi_i^{old}(a_i^k | o_i^k)}, \quad (192)$$

$$\mathcal{L}_i^{\text{clip}} = \mathbb{E} [\min (\rho_i A_i, \text{clip}(\rho_i, 1 - \epsilon, 1 + \epsilon) A_i)]. \quad (193)$$

For cyber games, MAPPO is often easier to stabilize than MADDPG when actions are discrete or hybrid.

G.6 Common MADRL failure modes in cyber games

1. **Reward domination by cost or damage.** If compromise loss is 1000 times larger than action cost, the learned agent may always choose maximum defense. Normalize terms.
2. **Opponent overfitting.** A defender trained against one scripted attacker may fail against a different attacker. Use opponent pools.
3. **No equilibrium check.** Self-play convergence curves do not prove Nash equilibrium. Use unilateral retraining tests.

4. **Unrealistic observations.** If the defender observes the attacker's hidden state perfectly, the result may be too optimistic.
5. **Ignoring impulse semantics.** If an action is modeled as an impulse, it should create a state jump or an immediate cost; if it only changes a rate over the next interval, call it a discrete decision with continuous flow instead.

H. Appendix: Detailed Conversion of Continuous, Impulsive, and Hybrid Models to RL

This appendix gives implementation-level details for the conversion described in Section 5. The key idea is that RL does not directly optimize a continuous-time functional. Instead, the environment converts one decision into a transition and reward.

H.1 Case 1: continuous flow only

Suppose a defender chooses a discrete action a_k at time t_k , and this action sets a constant control parameter $u_k = \phi(a_k)$ during the interval $[t_k, t_{k+1})$. Then

$$x_{k+1} = \text{ODESolve}(f, x_k, u_k, \Delta t). \quad (194)$$

A minimal reward is

$$r_k = -\Delta t L(x_{k+1}, u_k), \quad (195)$$

while a more accurate reward integrates the running loss along the ODE trajectory:

$$r_k = - \sum_{j=0}^{M-1} h L(x_{k,j}, u_k), \quad h = \Delta t / M. \quad (196)$$

The second form is recommended when the state changes rapidly inside one decision interval.

H.2 Case 2: impulse followed by continuous flow

For an impulse action, the first operation is not ODE integration but a jump map:

$$y_k = G(x_k, a_k). \quad (197)$$

Then the post-jump state is integrated:

$$x_{k+1} = \text{ODESolve}(f, y_k, u_k, \Delta t). \quad (198)$$

The reward should penalize the instantaneous intervention:

$$r_k = -C_{\text{jump}}(x_k, a_k) - \sum_{j=0}^{M-1} h L(x_{k,j}, u_k). \quad (199)$$

In code, this suggests a clear environment structure:

$$\boxed{\text{decode action}} \rightarrow \boxed{\text{jump map}} \rightarrow \boxed{\text{ODE solver}} \rightarrow \boxed{\text{reward}}. \quad (200)$$

H.3 Case 3: hybrid discrete mode plus continuous intensity

A hybrid action $a_k = (m_k, v_k)$ requires two decoding operations:

$$m_k = \text{selected mode}, \quad v_k = \text{continuous intensity}. \quad (201)$$

The mode chooses which parameters or vector field are active. The intensity sets the magnitude. For example,

$$\rho_k = \rho_0 + \rho_{\max} v_k \quad \text{if } m_k = \text{patch}, \quad (202)$$

while

$$\iota_k = \iota_{\max} v_k \quad \text{if } m_k = \text{isolate}. \quad (203)$$

Parameterized-action RL treats the policy as two heads:

$$\pi(m, v|s) = \pi_m(m|s)\pi_v(v|s, m). \quad (204)$$

The first head is often a softmax over modes; the second head outputs a bounded continuous value through a sigmoid or a distribution such as a Gaussian followed by clipping.

H.4 Case 4: attacker-defender Markov game

For a game, the environment receives both actions:

$$(a_D^k, a_A^k) = ((m_D^k, v_D^k), (m_A^k, v_A^k)). \quad (205)$$

A centralized critic can use all joint information,

$$Q_D(s_k, a_D^k, a_A^k), \quad Q_A(s_k, a_D^k, a_A^k), \quad (206)$$

but the deployed defender policy may observe only defender-side information. This is the idea behind centralized training and decentralized execution.

H.5 Common mistakes

- Treating a discrete decision as an impulse when the model only changes the rate during the next interval.
- Ignoring impulse cost, which can make the learned policy use unrealistically strong emergency actions.
- Using $x(t_k^-)$ and $x(t_k^+)$ inconsistently. Always state whether the RL state is pre-jump or post-jump.
- Claiming that DQN solves PMP. DQN solves the discretized MDP induced by the continuous-time model; it does not solve the costate boundary-value problem unless PMP residuals or expert PMP trajectories are explicitly used.

H.6 Python skeleton: impulse and hybrid cyber environment

Listing 1: Impulse and hybrid ODE environment skeleton.

```

1 import numpy as np
2
3 ACTIONS = {
4     0: ("monitor", 0.5),
5     1: ("patch", 0.7),
6     2: ("isolate", 0.8),
7     3: ("deceive", 0.6),
8 }
9
10 def jump_map(x, mode, intensity):
11     S, I, R, z = x
12     if mode == "patch":
13         moved = intensity * 0.20 * S # emergency patch fraction
14         S, R = S - moved, R + moved
15     elif mode == "isolate":
16         moved = intensity * 0.30 * I # immediate isolation/cleaning
17         I, R = I - moved, R + moved
18     # monitor and deceive may have no instantaneous jump
19     return np.array([S, I, R, z], dtype=float)
20
21 def rhs(x, mode, intensity, params):
22     S, I, R, z = x
23     beta, gamma, omega, chi = params
24     patch_rate = clean_rate = deceive_rate = 0.0
25     if mode == "monitor":
26         clean_rate = 0.20 * intensity
27     elif mode == "patch":
28         patch_rate = 0.15 * intensity
29     elif mode == "deceive":
30         deceive_rate = 0.25 * intensity
31     effective_beta = beta * (1.0 - chi * z)
32     dS = -effective_beta * S * I - patch_rate * S + omega * R
33     dI = effective_beta * S * I - (gamma + clean_rate) * I
34     dR = patch_rate * S + (gamma + clean_rate) * I - omega * R
35     dz = deceive_rate * (1.0 - z) - 0.05 * z
36     return np.array([dS, dI, dR, dz], dtype=float)
37
38 def rk4_step(x, mode, intensity, params, dt):
39     k1 = rhs(x, mode, intensity, params)
40     k2 = rhs(x + 0.5 * dt * k1, mode, intensity, params)
41     k3 = rhs(x + 0.5 * dt * k2, mode, intensity, params)
42     k4 = rhs(x + dt * k3, mode, intensity, params)
43     return x + dt * (k1 + 2*k2 + 2*k3 + k4) / 6.0
44
45 def env_step(x, action_id, params, delta_t=1.0, substeps=20):
46     mode, intensity = ACTIONS[action_id]
47     y = jump_map(x, mode, intensity) # instantaneous part
48     h = delta_t / substeps
49     running_loss = 0.0
50     x_next = y.copy()
51     for _ in range(substeps):
52         running_loss += h * (10.0 * x_next[1] + 0.1 * intensity**2)

```

```

53     x_next = rk4_step(x_next, mode, intensity, params, h)
54     x_next[:3] = np.maximum(x_next[:3], 0.0)
55     x_next[:3] = x_next[:3] / max(x_next[:3].sum(), 1e-8)
56     x_next[3] = np.clip(x_next[3], 0.0, 1.0)
57     jump_cost = 0.5 * intensity**2 if mode in ["patch", "isolate"] else 0.0
58     reward = -(running_loss + jump_cost)
59     return x_next, reward

```

I. Python Skeletons for ODE-RL and DDQN

This appendix gives compact Python-style skeleton code. It is not intended to be a polished library. Its purpose is to make the implementation mapping explicit:

ODE equations $\rightarrow$ environment step $\rightarrow (s, a, r, s') \rightarrow$ DQN/DDQN update.

I.1 Minimal RK4-based cyber defense environment

The following environment uses a controlled malware model with three compartments S, I, R . The action set is discrete: no operation, patch, clean, and strong response. The function `action_to_control` implements the action-to-control map $\Gamma(a)$ introduced in the main text.

Listing 2: Minimal ODE-based cyber defense environment.

```

1  import numpy as np
2
3  class MalwareODEEnv:
4      def __init__(self, dt=1.0, horizon=100):
5          self.dt = dt
6          self.horizon = horizon
7          self.beta = 0.60 # infection rate
8          self.gamma = 0.05 # natural recovery rate
9          self.omega = 0.01 # loss of protection rate
10         self.max_patch = 0.40
11         self.max_clean = 0.35
12         self.cost_patch = 0.8
13         self.cost_clean = 1.0
14         self.w_infected = 10.0
15         self.k = 0
16         self.x = None
17
18     def reset(self, x0=None):
19         if x0 is None:
20             x0 = np.array([0.95, 0.05, 0.0], dtype=np.float64)
21             self.x = x0.copy()
22             self.k = 0
23             return self.observe()
24
25     def observe(self):
26         # In practice, include budget, alerts, time-to-go, or noisy estimates.
27         return self.x.copy()
28
29     def action_to_control(self, action):

```

```

30     # action 0: no-op, 1: patch, 2: clean, 3: strong response
31     if action == 0:
32         return 0.0, 0.0
33     if action == 1:
34         return self.max_patch, 0.05
35     if action == 2:
36         return 0.05, self.max_clean
37     if action == 3:
38         return 0.7 * self.max_patch, 0.7 * self.max_clean
39     raise ValueError("unknown action")
40
41     def rhs(self, x, u_patch, u_clean):
42         S, I, R = x
43         dS = -self.beta * S * I - u_patch * S + self.omega * R
44         dI = self.beta * S * I - self.gamma * I - u_clean * I
45         dR = self.gamma * I + u_patch * S + u_clean * I - self.omega * R
46         return np.array([dS, dI, dR], dtype=np.float64)
47
48     def rk4_step(self, x, u_patch, u_clean):
49         h = self.dt
50         k1 = self.rhs(x, u_patch, u_clean)
51         k2 = self.rhs(x + 0.5 * h * k1, u_patch, u_clean)
52         k3 = self.rhs(x + 0.5 * h * k2, u_patch, u_clean)
53         k4 = self.rhs(x + h * k3, u_patch, u_clean)
54         x_next = x + (h / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
55         x_next = np.clip(x_next, 0.0, 1.0)
56         x_next = x_next / max(x_next.sum(), 1e-12)
57         return x_next
58
59     def step(self, action):
60         u_patch, u_clean = self.action_to_control(action)
61         x_next = self.rk4_step(self.x, u_patch, u_clean)
62         S, I, R = x_next
63         running_loss = self.w_infected * I
64         control_cost = self.cost_patch * u_patch**2 + self.cost_clean * u_clean**2
65         reward = -self.dt * (running_loss + control_cost)
66         self.x = x_next
67         self.k += 1
68         done = (self.k >= self.horizon) or (I < 1e-4) or (I > 0.95)
69         return self.observe(), reward, done, {"u_patch": u_patch, "u_clean": u_clean}

```

I.2 Environment sanity checks before training

Before training an agent, check no-defense, static patching, static cleaning, and strong-response rollouts. If these baselines do not behave sensibly, the RL result will not be meaningful.

Listing 3: Simple environment sanity check.

```

1 def rollout(env, policy_fn):
2     s = env.reset()
3     traj, rewards = [s], []
4     done = False
5     while not done:
6         a = policy_fn(s)
7         s, r, done, info = env.step(a)

```

```

8     traj.append(s)
9     rewards.append(r)
10    return np.array(traj), np.array(rewards)
11
12    env = MalwareODEEnv()
13    traj_no_def, rew_no_def = rollout(env, lambda s: 0)
14    traj_patch, rew_patch = rollout(env, lambda s: 1)
15    traj_clean, rew_clean = rollout(env, lambda s: 2)
16    print("Peak I no defense:", traj_no_def[:, 1].max())
17    print("Peak I patch:", traj_patch[:, 1].max())
18    print("Peak I clean:", traj_clean[:, 1].max())

```

I.3 DQN network and DDQN target update

DQN approximates $Q(s, a)$ for discrete actions. DDQN reduces overestimation by selecting the next action using the online network but evaluating it using the target network.

Listing 4: DQN/DDQN update skeleton in PyTorch.

```

1  import torch
2  import torch.nn as nn
3  import torch.nn.functional as F
4
5  class QNet(nn.Module):
6      def __init__(self, state_dim=3, n_actions=4, hidden=128):
7          super().__init__()
8          self.net = nn.Sequential(
9              nn.Linear(state_dim, hidden), nn.ReLU(),
10             nn.Linear(hidden, hidden), nn.ReLU(),
11             nn.Linear(hidden, n_actions)
12         )
13     def forward(self, s):
14         return self.net(s)
15
16     def ddqn_loss(batch, q_online, q_target, gamma=0.99):
17         s, a, r, s_next, done = batch
18         q_values = q_online(s).gather(1, a.long().view(-1, 1)).squeeze(1)
19         with torch.no_grad():
20             next_action = q_online(s_next).argmax(dim=1, keepdim=True)
21             next_q = q_target(s_next).gather(1, next_action).squeeze(1)
22             target = r + gamma * (1.0 - done) * next_q
23         return F.mse_loss(q_values, target)

```

I.4 Minimal training loop structure

The loop below omits replay-buffer implementation details. The important point is that each replay item is generated by integrating the ODE for one decision interval.

Listing 5: DQN training loop structure for an ODE environment.

```

1  q_online = QNet()
2  q_target = QNet()
3  q_target.load_state_dict(q_online.state_dict())
4  optimizer = torch.optim.Adam(q_online.parameters(), lr=1e-3)

```

```

6
7 epsilon = 1.0
8 for episode in range(500):
9     s = env.reset()
10    done = False
11    while not done:
12        if np.random.rand() < epsilon:
13            a = np.random.randint(4)
14        else:
15            with torch.no_grad():
16                ss = torch.tensor(s, dtype=torch.float32).unsqueeze(0)
17                a = int(q_online(ss).argmax(dim=1).item())
18        s_next, r, done, info = env.step(a)
19        replay_buffer.add(s, a, r, s_next, float(done))
20        s = s_next
21        if replay_buffer.size() >= 1000:
22            batch = replay_buffer.sample(batch_size=64)
23            loss = ddqn_loss(batch, q_online, q_target, gamma=0.99)
24            optimizer.zero_grad()
25            loss.backward()
26            optimizer.step()
27        epsilon = max(0.05, epsilon * 0.995)
28    if episode % 10 == 0:
29        q_target.load_state_dict(q_online.state_dict())

```

I.5 Extension to attacker-defender Markov games

For a Markov game, the environment step accepts two actions. The ODE right-hand side depends on both. The replay item should store joint information if using centralized training and decentralized execution.

Listing 6: Attacker-defender environment step interface.

```

1 def step_markov_game(state, attacker_action, defender_action):
2     uA = attacker_action_to_control(attacker_action)
3     uD = defender_action_to_control(defender_action)
4     next_state = integrate_cyber_ode(state, uA, uD)
5     rD = defender_reward(next_state, uA, uD)
6     rA = attacker_reward(next_state, uA, uD)
7     return next_state, rA, rD
8
9 # CTDE critic can use joint action information during training:
10 # Q_D(s, a_D, a_A), Q_A(s, a_D, a_A).
11 # During execution, each actor may use only its own observation.

```

How to present Python code in a paper. Do not include long engineering code in the main paper. Present the environment equations, action map, reward definition, algorithm pseudocode, and implementation settings. Put full code in a repository. In a lecture note or tutorial, skeleton code like the above is useful because it shows exactly how mathematical notation becomes a simulator.

References

- [1] L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko, *The Mathematical Theory of Optimal Processes*. Wiley, 1962.
- [2] S. Lenhart and J. T. Workman, *Optimal Control Applied to Biological Models*. Chapman and Hall/CRC, 2007.
- [3] M. McAsey, L. Mou, and W. Han, “Convergence of the forward-backward sweep method in optimal control,” *Computational Optimization and Applications*, vol. 53, pp. 207–226, 2012.
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [5] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [6] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proceedings of AAAI*, 2016.
- [7] T. P. Lillicrap et al., “Continuous control with deep reinforcement learning,” arXiv:1509.02971, 2015.
- [8] J. Schulman, P. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [9] T. Haarnoja et al., “Soft actor-critic algorithms and applications,” arXiv:1812.05905, 2018.
- [10] M. L. Littman, “Markov games as a framework for multi-agent reinforcement learning,” in *Proceedings of the 11th International Conference on Machine Learning*, pp. 157–163, 1994.
- [11] R. Lowe et al., “Multi-agent actor-critic for mixed cooperative-competitive environments,” arXiv:1706.02275, 2017.
- [12] C. Yu et al., “The surprising effectiveness of PPO in cooperative multi-agent games,” arXiv:2103.01955, 2021.
- [13] M. Hausknecht and P. Stone, “Deep reinforcement learning in parameterized action space,” arXiv:1511.04143, 2015.
- [14] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, “Neural ordinary differential equations,” *Advances in Neural Information Processing Systems*, 2018.
- [15] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations,” *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019.
- [16] J. Sirignano and K. Spiliopoulos, “DGM: A deep learning algorithm for solving partial differential equations,” *Journal of Computational Physics*, vol. 375, pp. 1339–1364, 2018.
- [17] L.-X. Yang, P. Li, Y. Zhang, X. Yang, Y. Y. Tang, and Y. Xiang, “Effective repair strategy against advanced persistent threat: A differential game approach,” *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 7, pp. 1713–1728, 2019.

- [18] X. Cheng et al., “Cost-effective hybrid control strategies for dynamical propaganda war game,” *IEEE Transactions on Information Forensics and Security*, 2024.
- [19] H. Sun et al., “Impulsive artificial defense against advanced persistent threat,” *IEEE Transactions on Information Forensics and Security*, 2023.
- [20] Y. Qin, X. Yang, L.-X. Yang, and K. Huang, “Modeling and study of defense outsourcing against advanced persistent threat through impulsive differential game approach,” *Computers & Security*, vol. 145, 104003, 2024.
- [21] S. Shen, L. Xie, Y. Zhang, G. Wu, H. Zhang, and S. Yu, “Joint differential game and double deep Q-networks for suppressing malware spread in Industrial Internet of Things,” *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 5302–5316, 2023.
- [22] W. He, J. Tan, R. Wang, Z. Liu, X. Luo, H. Hu, and H. Zhang, “A deep reinforcement learning approach to time delay differential game deception resource deployment,” *IEEE Transactions on Dependable and Secure Computing*, vol. 23, no. 1, pp. 1655–1670, 2026.
- [23] W. He, J. Tan, Y. Guo, K. Shang, and H. Zhang, “A deep reinforcement learning-based deception asset selection algorithm in differential games,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 8353–8366, 2024.
- [24] S. Shen et al., “Privacy-aware DRL for differential games-assisted malware defense in edge intelligence-enabled Social IoT,” *IEEE Transactions on Network and Service Management*, vol. 23, pp. 2680–2695, 2026.
- [25] Y. Shen, C. Shepherd, C. M. Ahmed, S. Shen, and S. Yu, “Malware patching strategies in edge intelligence IoT systems: A differential games approach with spiking neural networks,” *IEEE Transactions on Dependable and Secure Computing*, early access, 2026.
- [26] L. Zhang, M. Ghimire, Z. Xu, W. Zhang, and Y. Ren, “Pontryagin neural operator for solving general-sum differential games with parametric state constraints,” in *Proceedings of Learning for Dynamics and Control*, PMLR, vol. 242, pp. 1–13, 2024.
- [27] D. Han, S. Jin, and C. Li, “Controlling epidemics with information dynamics: A Neural ODE approach,” *IEEE Transactions on Computational Social Systems*, vol. 12, no. 6, pp. 4179–4192, 2025.
- [28] Y. Hou et al., “SAPDR: A self-adaptive physics-informed deep learning framework for rumor propagation dynamics,” 2026.
- [29] Y. Jing et al., “Physics-informed learning for simplicial complex rumor dynamics,” 2025.
- [30] L. Ma et al., “Physics-informed learning and optimal control for negative information propagation,” 2026.
- [31] R. Isaacs, *Differential Games: A Mathematical Theory with Applications to Warfare and Pursuit, Control and Optimization*. Wiley, 1965.
- [32] T. Basar and G. J. Olsder, *Dynamic Noncooperative Game Theory*, 2nd ed. SIAM, 1999.